QSC SDK C Examples and Qualification Platform – QSCSDKC

Technical Specification · Supplied Source Snapshot · November 9, 2026

Quantum Resistant Cryptographic Solutions Corporation

This engineering specification describes two related native-C products: the QSCSDKC interactive cryptographic example console, and the QSCSDKTest standalone local cryptographic qualification platform. It provides implementation architecture, native QSC integration, CLI and web APIs, testing and evidence models, source-grounded public function references, example commands, test catalogs, and validation-vector provenance.

All source and non-public technical details remain proprietary to QRCS. The source carries the QRCS Commercial Application Software License (QRCS-CASL) notice; the controlling license and any commercial agreement govern use. This specification is not a redistribution authorization.

Contents

1 Foreword 3
2 Introduction and Supplied Source Baseline 3
3 References and Engineering Conventions 4
4 System Architecture and Native Dependency 5
5 Build and Integration 6
6 QSCSDKC Console Lifecycle and Commands 7
7 Classical and Post-Quantum Demonstrations 8
8 Hash, MAC, KDF and DRBG Demonstrations 9
9 AES, ChaCha, CSX and RCS Demonstrations 9
10 Utility and Platform Demonstrations 10
11 TLS 1.3 and X.509 Examples 10
12 QSCSDKTest Qualification Architecture 11
13 Test Registry, Assertions and Evidence Classes 11
14 Packaged NIST Response Vectors 12
15 Cryptographic and TLS Validation Tests 12
16 Benchmark Design and CPU Profile Reporting 13
17 Fuzz Testing and Process Isolation 13
18 Asynchronous Run Manager and Cancellation 13

19 Local Browser Workbench and Session Security 14
20 HTTP and JSON Endpoint Contract 15
21 Evidence Reports, Exports and Integrity 15
22 CLI Workflows and Operator Guide 16
23 Storage, Paths, Cross-Platform Portability 17
24 Security and Correctness Boundaries 17
25 Test and Release Acceptance Plan 18
26 QSCSDKC Public C Function Reference 19
27 QSCSDKTest Public C Function Reference 74
Annex A Complete Interactive Command Catalog 91
Annex B Registered Qualification Tests 101
Annex C Packaged Vector Corpus 102
Annex D Public Types, Enums and Constants 104

1: Foreword

QSCSDKC is the C-language example application for the external QSC cryptographic library. It demonstrates complete call sequences for classical and post-quantum cryptography, authenticated ciphers, key derivation, entropy providers, TLS and X.509. The companion QSCSDKTest application is a separate qualification product: it registers specifically named validation cases, executes them, records structured assertions, manages local benchmark and fuzz jobs, and exports machine- and human-readable evidence.

The two applications are related but not interchangeable. The examples console teaches invocation and tests basic roundtrips or negative cases; QSCSDKTest is a locally operated evidence workbench. Neither application is itself a replacement for QSC cryptographic library APIs, nor is a passing local test a CAVP/ACVP certificate or an external audit.

This specification follows the QRCS QSC reference layout: US Letter, 1-inch margins, Segoe UI body and Segoe UI Semibold headings, no bold font runs, black text except hyperlinks, numbered chapters and complete API/command catalogs, with tables aligned to the maximum paragraph width.

2: Introduction and Supplied Source Baseline

2.1 Archive and project organization

The uploaded QSCSDKC.zip contains a Visual Studio solution, the QSCSDKC application directory and the QSCSDKTest application directory. The solution lists QSC and QSCSDKC, while QSCSDKTest supplies its own Visual Studio project that is not included in the supplied solution's project list. Both applications include the external QSC headers and/or project by relative path; QSC source itself is not packaged in this archive.

Source property	Verified archive value
SDK examples	QSCSDKC/ contains 45 .c files and 47 headers, including sdkc.c, sdkcutils.c and per-family demonstrations.
Qualification workbench	QSCSDKTest/ contains 31 .c files and 31 headers, plus local vector files, web assets and a standalone Visual Studio project.
Declared C interface	532 prototype declarations across 73 contributing headers: 401 in QSCSDKC and 131 in QSCSDKTest.
Example CLI	393 distinct command strings in the interactive sdkc.c dispatcher.
Qualification catalog	26 statically registered test definitions across selftest, crypto, TLS, vector and assurance categories.
Packaged test vectors	72 verified files in 6 algorithm families; the manifest declares 11,103 cases.
Native dependencies	Direct native QSC C library headers and build target, not QSCABI, Rust, Python or .NET.

Source property	Verified archive value
Build evidence	No C/MSVC/GCC/Clang build or executable run performed for this specification; static source and vector checks only.
License	Commercial proprietary QRCS-CASL notice present in the uploaded source headers.

Table 2.1 Uploaded source and qualification baseline

2.2 Document scope and source precedence

Source declarations, documented macros, selected test registry records and the supplied vector manifest are used as authority. A command string in sdkc.c is a demonstration dispatch token; it is not a new native QSC function. A test registry entry is the description of a check the platform can run; it is not evidence that the check has already passed on the user's machine.

3: References and Engineering Conventions

3.1 Primary project files

- QSCSDKC/sdkc.c, sdkc.h, sdkccommon.h and sdkcutils.h — interactive application, status and terminal I/O.
- QSCSDKC/help.c, help.h — command help and user-visible command descriptions.
- QSCSDKC/*_example.h and *_example.c — algorithms, file/network helpers, resource lifetime and individual demonstrations.
- QSCSDKTest/qscsdk_program.c and qscsdk_cli.* — command-line actions and program dispatch.
- QSCSDKTest/qscsdk_registry.* — immutable test definitions, test IDs, descriptions and evidence flags.
- QSCSDKTest/qscsdk_common.h and qscsdk_result.* — records, statuses, assertions and typed result construction.
- QSCSDKTest/qscsdk_run_manager.*, qscsdk_runner.* — asynchronous work and direct single-test execution.
- QSCSDKTest/qscsdk_benchmark.* and qscsdk_fuzz.* — performance and bounded adversarial test managers.
- QSCSDKTest/qscsdk_web_server.*, qscsdk_web_platform.*, qscsdk_web_session.* — HTTP loopback serving and browser authorization.
- QSCSDKTest/qscsdk_report.*, qscsdk_evidence.*, qscsdk_executive_report.* and qscsdk_zip.* — generated evidence.
- QSCSDKTest/Vectors/manifest.json and algorithm-specific .rsp files — immutable response-vector provenance.

3.2 External standards

QSC library public source — https://github.com/Steppenwolfe65/QSC

NIST CAVP algorithm validation overview — https://csrc.nist.gov/projects/cryptographic-algorithm-validation-program

NIST FIPS 197 AES — https://doi.org/10.6028/NIST.FIPS.197-upd1

NIST FIPS 202 SHA-3 — https://doi.org/10.6028/NIST.FIPS.202

NIST FIPS 203 ML-KEM — https://doi.org/10.6028/NIST.FIPS.203

NIST FIPS 204 ML-DSA — https://doi.org/10.6028/NIST.FIPS.204

NIST FIPS 205 SLH-DSA — https://doi.org/10.6028/NIST.FIPS.205

RFC 8446 TLS 1.3 — https://www.rfc-editor.org/rfc/rfc8446

RFC 5280 X.509 certificates — https://www.rfc-editor.org/rfc/rfc5280

3.3 Result and terminology conventions

Public qscsdkc_status is an application-level success/failure type; qscsdk_test_status is a separate four-valued PASS/FAIL/SKIP/ERROR test outcome. A qscsdk_test_definition is immutable metadata and a function pointer, while a qscsdk_test_result records observed status and detailed evidence. An environment summary describes the tested host/build, not a proof of CPU-target safety. Byte counts and output capacities must be interpreted according to each C declaration.

4: System Architecture and Native Dependency

4.1 Two application call graphs

```
QSCSDKC.exe (C examples console)
  sdkc.c command dispatcher -> *_example.c modules
  -> native QSC C headers/functions


QSCSDKTest.exe (qualification workbench)
  qscsdk_program.c CLI -> test registry / single-test runner
  -> QSC C selftests, vector tests, TLS/X509 checks
  -> assertions / environment / JSON and HTML evidence
  -> optional HTTP loopback workbench with background run managers
```

4.2 Native QSC and binary profiles

Both SDK executables compile against the external QSC C project, using direct include and link dependencies. Unlike language wrappers QSCCS/QSCRS/QSCPY, the SDK applications

do not use the QSCABI frozen 329-export interface. QSC runtime behavior and supported KEM/signature parameter selections are determined by the linked QSC build, compiler, processor features and compile-time macros.

4.3 Internal module boundaries

Component	Role	Trust / ownership boundary
QSCSDKC sdkc.c	Operator command parsing, command routing and overall example execution.	Consumes user-provided console tokens; each named module returns independent status.
QSCSDKTest registry	Immutable metadata and function pointers.	Test definitions enumerate available cases; no network code in registration itself.
Runner/result	Invokes one test, captures assertions, elapsed time and status.	No success presumed from existence of a declaration.
Run manager	Owns job lifecycle, cancellation state and bounded history.	Threaded access and snapshots require valid owner and cleanup.
Browser workbench	Loopback HTTP server, session, JSON and static assets.	Local session token and Origin validation gate protected actions.
Evidence pipeline	Reports, manifests, artifacts and zip bundles.	Path and size validation; hash provenance must be checked by recipient.

Table 4.1 SDK component responsibilities

5: Build and Integration

5.1 Windows Visual Studio project model

QSCSDKC.sln includes a QSC native project reference and the QSCSDKC console executable. The C example project references ../../QSC/QSC.vcxproj. QSCSDKTest.vcxproj is provided independently and references a relative QSC project that must be verified/reconciled against the developer checkout. Both Debug and Release configurations occur in the Visual Studio metadata, with platform profiles varying by project. Ensure all dependent native QSC sources, headers, include directories, CRT selection and instruction-set flags agree before release.

5.2 Non-Windows build integration

The archive does not supply a Unix Makefile or complete CMake project for the two executables. Linux/macOS integration therefore requires a separately authorized build definition with the required project source lists, QSC dependencies, socket/thread libraries, platform guards and target ISA. A platform path in the code should not be taken as evidence that this snapshot was built or qualified on that target.

5.3 Project-specific compile considerations

QSCSDKTest provides OS-dependent local web sockets, monotonic timing, random token generation, filesystem reports and concurrent work managers. Guarded platform implementations and native data-type widths must be reviewed on each OS. QSC optimized machine-code variants cannot be selected safely from a label-only CPU banner; the binary itself must match supported host instructions.

5.4 Suggested build dependency order

External QSC native library (matching source and CPU profile)
-> QSCSDKC examples console
-> QSCSDKTest standalone qualification application
-> local vector corpus under its expected Vectors/ directory
-> command line, then browser and multi-host acceptance

6: QSCSDKC Console Lifecycle and Commands

6.1 Operator entry points

sdkc.c implements a fixed sequence for -run all, a named command selector tree, help/list handling and a quit path. Tokens are command strings such as -sha2, -aes, -mlkem and -x509 with subcommands. The source contains 393 distinct exact strings, reproduced in Annex A. Missing or incorrect parameters are not silently interpreted as another cipher or parameter set.

```
# Launch QSCSDKC.exe with no arguments; enter commands at its prompt:
-list
-sha2
-aes
-mlkem keypair
-run all
-quit
```

The QSCSDKC source uses `int main(void)` and obtains each line with `qsc_consoleutils_get_line`; it does not parse process arguments. These tokens must be entered at the interactive console prompt. At shutdown, the source displays a Press-any-key prompt, which may block automation. File, entropy, TLS and other examples can require suitable environment or peers. The separate QSCSDKTest application offers argument-driven and noninteractive execution.

6.2 Sequential execution behavior

The `qscsdkc_run_all()` routine chains module run_all results through a single Boolean variable. Its `&&` short-circuiting means later function calls may be skipped after the first reported failure. That differs materially from QSCSDKTest, which has a structured registry with per-test outcomes and evidence. The Falcon examples are present in source but their invocation is commented out in the all-tests sequence pending the project's stated certification policy. CJP and RDP examples are called only if their runtime availability predicates succeed.

6.3 Status, informational header and clean exit

sdkcutils.* handles command comparison, status display, CPU/intrinsics information and banner utilities. `sdkccommon.h` defines project status macros/types and compile-time visibility/debug controls. Standalone interactive behavior depends on command and terminal; do not mistake an optional non-CI close prompt for a cryptographic requirement.

7: Classical and Post-Quantum Demonstrations

7.1 KEM and curve groups

The examples cover ECDH and EDDH reciprocal agreement, ML-KEM, HQC and Classic McEliece KEMs, parameter-relevant keypair/encapsulation/decapsulation operations, shared-secret derivation, role exchange, public-key representation, file storage, verification and negative cases. KEM decapsulation does not independently authenticate the peer; every real protocol must bind peer identity, parameter identifier and transcript to resulting traffic keys.

7.2 Signature groups

ML-DSA, ECDSA, EdDSA, Falcon and SPHINCS+ groups demonstrate keypairs, signing, verification, roles, invalid input and selected file operations. Algorithm availability and parameter widths follow the linked QSC source. The example runner intentionally omits automatic Falcon execution under its stated certification policy; no independently verified Falcon certification status is asserted here.

7.3 Security-relevant negative checks

Negative-path demonstrations are critical for acceptance: wrong key, wrong ciphertext length, bad signature, invalid seed, modified authentication data or incorrect algorithm selection must fail rather than silently produce an accepted result. An example function

name ending in `_negative` is only an intended check; the actual result must be inspected in a new execution.

8: Hash, MAC, KDF and DRBG Demonstrations

SHA-2/SHA-3, SHAKE/cSHAKE, HMAC, KMAC, PBMAC1, QMAC and HKDF demonstrations exercise static and stateful computations against QSC. Hash outputs authenticate nothing by themselves; MAC verification requires the exact key and domain separation. PBMAC1 salt and iteration policy is a deployment parameter, and HKDF info must bind the intended protocol purpose.

The CSG, HCG, SCB and SECRAND modules demonstrate stateful generators, seed/reseed requirements and native error reporting. ACP, CSP, CJP and RDP cover distinct native entropy sources; CJP and RDP are conditionally exercised only where the backend reports availability. A test vector or fixed seed is acceptable in deterministic evidence, not a production key-generation substitute.

9: AES, ChaCha, CSX and RCS Demonstrations

The AES demonstration includes encryption/decryption, ECB/CBC/CTR, authenticated GCM/HBA and relevant padding, IV and counter behavior. ChaCha20 and ChaCha20-Poly1305 are presented separately; unauthenticated ChaCha/CTR output requires independent integrity protection. CSX and RCS demonstrate QRCS authenticated ciphers, associated data, file/buffer operations and nonce lifecycles. Never reuse a nonce with the same authenticated cipher key.

Extended or streaming authenticated operations that release plaintext before final tag verification must quarantine all such plaintext until authentication succeeds, and destroy/discard it on failure. Do not substitute an example roundtrip for full known-answer and cross-implementation validation.

10: Utility and Platform Demonstrations

The project includes encoding, memory/integer/string/file utilities, timestamp conversion, CPU detection, collections and socket helpers. These are demonstration calls into QSC utility modules rather than a new implementation of QSC memory safety. File examples must respect writable user directories and reject unsafe relative paths; memory examples must distinguish constant-time equality from ordinary comparisons.

CPU feature output describes a host's reported capabilities, not the feature path compiled into QSC binaries. Socket utility demonstrations and TLS live-client/server examples need suitable networking and local permissions; they should not open unintended external endpoints without an operator choice.

11: TLS 1.3 and X.509 Examples

11.1 TLS transport

tls_example.c demonstrates certificate configuration, client/server operations, record handling, peer authentication, and a separate live TLS execution route. The `qscsdkc_tls_run_all()` and `qscsdkc_tls_live_run_all()` entry points are called from the aggregate example function. Because a live peer may be required, record exact role, trust anchors, hostname, network addresses and cipher selections in any acceptance log.

11.2 X.509 and issuance

The x509_example and x509_author_example modules cover certificate parsing/inspection, CSR issuance, CA/root tasks, PEM conversions, keys and verification. Successful parsing, signature checking or CSR generation alone cannot establish issuer authorization or trust. A deployed certificate workflow must verify hostname, trust roots, path constraints, validity, intended use and revocation according to the relying-party policy.

12: QSCSDKTest Qualification Architecture

12.1 Entry point and default action

`qscsdk_program.c` delegates to `qscsdk_cli_parse`, then dispatches help, catalog, list, all-tests, selected test/category, fuzz worker or browser server mode. With no CLI arguments, the default action is the local browser workbench. The single-test runner invokes one registered qscsdk_test_function and stores the result with environment and detail metadata.

12.2 Assertion and result model

Each test receives a `const qscsdk_test_context*`, a detail buffer and its capacity, and returns a four-state `qscsdk_test_status`. The result model adds per-assertion records, output artifacts, diagnostics and links to test metadata. Assertions marked INFO are evidence metadata, not passing assertions. The four terminal results remain PASS, FAIL, SKIP and ERROR and must not be conflated.

Status	Meaning and report rule
QSCSDK_TEST_STATUS_PASS	Requested test execution completed and its asserted conditions passed.

Status	Meaning and report rule
QSCSDK_TEST_STATUS_FAIL	An expected cryptographic or structural requirement was not met.
QSCSDK_TEST_STATUS_SKIP	Test could not run under its prerequisites; not a pass.
QSCSDK_TEST_STATUS_ERROR	An execution, platform, file or internal error prevented reliable qualification.

Table 12.1 Four-state outcome contract

13: Test Registry, Assertions and Evidence Classes

The immutable `QSCSDK_REGISTRY` defines 26 named tests and their purposes, source functions, applicable standards, evidence source flags, expected result descriptions and execution type. Each registered test has a stable dotted ID. Consumers must preserve these IDs in JSON, HTML and report bundles so regression baselines can be compared without relying on human-readable display strings.

Evidence flags include SELF_TEST, FUNCTIONAL, KNOWN_ANSWER, STANDARD_VECTOR, SPECIFICATION, NEGATIVE, ROUND_TRIP, STRESS, FUZZ and BENCHMARK. A flag describes the intended kind of evidence, not that it was executed. The registry categories in this source are 6 selftests, 6 cryptographic checks, 6 TLS checks, 1 packaged-vector integrity check, and 7 standards-vector tests.

14: Packaged NIST Response Vectors

14.1 Manifest schema and source provenance

Vectors/manifest.json declares `qsc-sdk-vector-manifest-v1`, a provider note, source URL, file SHA-256, byte size, number of cases and family for each response-vector file. Its manifest explicitly states that passing packaged CAVP/ACVP material is informal implementation evidence and is not a formal CAVP/ACVP validation certificate.

Family	Files	Manifest cases
AES	28	3,674
MLDSA	6	120
MLKEM	6	148
SHA2	12	3,118
SHA3	14	3,992
SLHDSA	6	51

Table 14.1 Packaged vector families and case totals

14.2 Integrity evidence obtained in this preparation

Every one of the 72 local .rsp vector files was opened and its actual byte size and SHA-256 digest compared against the corresponding manifest values. All records matched. This does

not execute any cryptographic known-answer operation and does not verify that every declared case is consumed by a registered test. The full manifest table appears in Annex C.

14.3 Parameter-set selection and operational coverage

The test registry selects ML-KEM, ML-DSA and SLH-DSA vector files using native QSC compile-time parameter guards. Hence having several parameter-set .rsp files in a ZIP does not mean a single executable exercises all of them. Compare each run's build configuration and selected native parameters with the intended standards scope, then test every required build independently.

15: Cryptographic and TLS Validation Tests

The 26-case qualification catalog includes QSC internal selftests, PQC operations and tamper cases, ECC/EdDSA operations, TLS codec/extensions/transcript/record/Finished/PQC metadata, corpus integrity, and representative NIST AES/SHA/ML-KEM/ML-DSA/SLH-DSA vectors. Test functions accept an explicit result detail buffer; failures and errors should include the expected/observed condition without writing secrets to logs.

The existence of a registered TLS codec or transcript test does not prove a real TLS 1.3 handshake against a remote interoperable peer, nor does it prove a trusted certificate chain. End-to-end TLS identity and multi-host tests belong to a separate deployment qualification gate.

16: Benchmark Design and CPU Profile Reporting

The benchmark manager registers named definitions with algorithm/component, operation, metric, message size and iteration count. Its snapshot records whether a benchmark completed, elapsed microseconds, total processed bytes, operations, MiB/s throughput and operations per second. Results must be tied to hardware/compiled CPU capability flags, Debug/Release mode, single-thread load, message size and iteration profile.

A benchmark is performance evidence, not a cryptographic correctness test or a guarantee of ISA compatibility. The state machine supports idle, running, cancel-pending, completed, cancelled and error states; changing a benchmark profile while one is active must not reuse stale result buffers.

17: Fuzz Testing and Process Isolation

The fuzz manager supplies named targets, corpus paths, maximum input sizes, a deterministic seed, worker iteration limits and execution statuses. Its state includes IDLE,

RUNNING, CANCEL_PENDING, COMPLETED, CRASHED, CANCELLED and ERROR. `--fuzz-worker` is an internal subprocess action with explicit status path; an external process return alone must not be classified as PASS without inspecting the worker's bounded results and any crash artifact.

Fuzz cases should be repeatable from seed and saved corpus. Invalid ASN.1/X.509 encodings, truncation, hostile counts and memory allocation errors are expected fuzz concerns. Isolate the worker from application secrets and do not allow a browser caller to supply arbitrary host file paths or executable names.

18: Asynchronous Run Manager and Cancellation

`qscsdk_run_manager` stores opaque state for a currently selected background test set and a bounded history. An operator may start all tests or select stable IDs, request cancellation, poll a status snapshot, retrieve result briefs and full results, and ask for a past run by run_id. The state model distinguishes CANCEL_PENDING from CANCELLED and ERROR; a request to cancel is not proof that the running cryptographic check has stopped.

The header sets `QSCSDK_RUN_HISTORY_MAX` to 8, `QSCSDK_RUN_LOG_MAX` to 256 and `QSCSDK_RUN_LOG_TEXT_MAX` to 192. Snapshots are bounded copies rather than ownership transfers of internal manager state. Respect the documented lifetimes of returned records and dispose managers only after jobs have terminated.

19: Local Browser Workbench and Session Security

19.1 Loopback server and bootstrap

The workbench opens a TCP listener on the loopback address and uses a default port of 59671. The program creates a cryptographically random one-use bootstrap token, prints/opens an initial bootstrap URL, validates the token with a length-checked constant-time byte comparison, and exchanges it for a session cookie. The session cookie is HttpOnly and SameSite=Strict. The workbench is designed for local authorization, not multi-user Internet hosting.

19.2 Origin and HTTP controls

Protected HTTP requests require the local session cookie. Mutating POST endpoints additionally validate Origin. Responses set Cross-Origin-Opener-Policy and Cross-Origin-Resource-Policy headers; the server also validates request paths and body constraints. These controls reduce browser abuse but do not constitute an externally audited hardened web

service. The server should remain loopback-bound, and operator use should avoid browser profiles where local site tokens are accessible to unrelated scripts.

19.3 Browser process ownership

`qscsdk_web_server_run` constructs the browser workbench, run manager, benchmark manager, fuzz manager and session. Closing the session requests shutdown and must be sequenced with manager disposal. On a report export, the backend must validate run_id and file names before serving local artifacts, preventing traversal or arbitrary file read.

20: HTTP and JSON Endpoint Contract

Method	Endpoint	Behavior
GET	/api/health	Status probe (before or outside protected route set).
GET	/api/catalog	Registered tests and categories.
GET	/api/summary	Current qualification summary.
GET	/api/run/status	Poll run status and progress.
GET	/api/run/history	Recent bounded run history.
GET	/api/run/result	Result detail for run/test.
POST	/api/run/start	Start selected tests or all.
POST	/api/run/cancel	Request cancellation.
GET	/api/benchmarks	Benchmark definitions and status.
POST	/api/benchmarks/start	Start selected benchmark.
POST	/api/benchmarks/cancel	Cancel benchmark.
GET	/api/fuzz	Fuzz target definitions and state.
POST	/api/fuzz/start	Start bounded fuzz job.
POST	/api/fuzz/cancel	Cancel fuzz job.
GET	/api/coverage	Source/feature coverage summary.
GET	/api/reports	List generated evidence.
POST	/api/reports/generate	Generate report for completed run.
POST	/api/reports/export	Build exportable evidence bundle.
GET	/api/reports/file	Retrieve safe recognized report file.
GET	/api/session/status	Check session state.
POST	/api/session/close	Close browser session and request shutdown.

Table 20.1 Browser JSON routes observed in qscsdk_web_server.c

The backend uses source-defined HTTP JSON payloads (qscsdk_web_json.*). Exact request members and response field names must be matched to the compiled server implementation, rather than inferred from endpoint labels. It is not a public remote network API. Invalid session, Origin, path, request length or state must return an error and must not start a job.

21: Evidence Reports, Exports and Integrity

21.1 Report schemas and artifacts

`qscsdk_common.h` defines the test report schema `qsc-sdk-test-report-v2` and catalog schema `qsc-sdk-test-catalog-v1`. `qscsdk_evidence.h` declares the evidence-bundle schema `qsc-sdk-evidence-bundle-v3`. The export process produces qualification JSON/HTML, an Executive HTML report, advanced-qualification JSON, a manifest with hashes, an evidence zip, and per-test artifacts when available.

Generated artifact	Source-defined file name / meaning
JSON result	qualification-report.json: structured per-test environment/outcome and assertion records.
HTML result	qualification-report.html: reader-friendly qualification evidence.
Executive HTML	<run-id>-Executive-Report.html: executive summary.
Advanced JSON	advanced-qualification.json: expanded source/test evidence.
Manifest	manifest.json: bundle member names, hashes and file sizes.
ZIP	evidence-bundle.zip: reproducible local export package.
Metadata	bundle.meta: persisted run ID and bundle facts.

Table 21.1 Evidence-bundle outputs

21.2 Report integrity and trust boundary

Evidence generated by a local program is a record of that program's reported actions. Hashing the files protects against undetected post-generation changes when a separately trusted digest is available; a local manifest hash does not itself authenticate the publisher or provide regulatory validation. The consumer must independently verify manifest integrity, signed distribution identity, native QSC version and intended test coverage.

22: CLI Workflows and Operator Guide

22.1 Qualification CLI commands

```
QSCSDKTest.exe                    (launch local browser workbench)
QSCSDKTest.exe --help
QSCSDKTest.exe --list
QSCSDKTest.exe --catalog-json <file>
QSCSDKTest.exe --all --report-json <file>
QSCSDKTest.exe --test QSC.SELFTEST.AES --report-json <file>
QSCSDKTest.exe --category tls --seed <unsigned-integer>
QSCSDKTest.exe --serve --port 59671 --no-browser
```

The CLI parser checks mutually exclusive actions and disallows options outside their intended action. For example, the port/browser switches belong to `--serve`, while `--report-json` belongs to test execution. `--fuzz-worker` is reserved for internal worker invocation and requires an explicit status path.

22.2 Failure and exit handling

CLI result codes distinguish successful action, invalid invocation, unavailable test or artifact, and execution/transport failure. The terminal runner uses a stable test ID, expected result field and error details in its output. Script automation should use process status and JSON, not parse colored console lines as a security decision.

23: Storage, Paths, Cross-Platform Portability

On Windows, the workbench uses `%LOCALAPPDATA%\QRCS\QSCSDKTest\Reports` for persistent run reports. The code has conditional non-Windows directory creation and pathname handling, but portable paths and writes must be confirmed for the target OS and permissions. The source deliberately validates run_id characters and rejects relative-path traversal on exports; application code must preserve those checks.

The bundled response vector files are read-only fixtures whose manifest paths begin under `Vectors/`; use them from the expected source/build root and do not replace them at runtime with downloaded unauthenticated test files. Error reporting, JSON escaping, file names, log text limits and safe artifact copy behavior are part of the security boundary. No deployment should write into a protected system folder or silently overwrite source fixtures.

24: Security and Correctness Boundaries

- The SDK examples and workbench call QSC directly; their declarations do not stabilize an external C ABI for third-party applications.
- Do not treat a successful demo roundtrip, one self-test or a vector-file integrity check as full algorithm qualification.
- On an invalid AEAD tag, plaintext must never be consumed before verification; require record nonce uniqueness.
- KEM output is not peer authentication and signatures prove only validity under the supplied public key.
- TLS internal unit tests are not a live, mutually authenticated TLS connection or certificate chain qualification.
- A detected AVX/AVX2/AVX-512 CPU flag does not select the correct compiled native QSC binary automatically.

- Local browser authorization is for loopback use; never expose HTTP endpoints to untrusted remote clients.
- Fuzz jobs must run in the bounded worker model and preserve invalid/crash/error distinctions.
- Evidence hashes prove local file integrity relative to their expected hash; they do not constitute official CAVP/ACVP certification.
- All QRCS source rights, reproduction rights, dependencies and distribution permissions remain controlled by QRCS-CASL and other applicable license agreements.

25: Test and Release Acceptance Plan

Qualification gate	Required recorded evidence
Build provenance	Target OS/compiler, build mode, source revisions, QSC external dependency, architecture and ISA.
Source interface	Audit all 532 extracted declarations and feature guards; confirm exported and internal API ownership.
Example console	Run representative command groups and all applicable explicit negative tests.
Catalog	All 26 registered test definitions accounted for; SKIP reported separately.
Standards vectors	Check corpus hash/size, execute selected vector parsers and require correct case counts.
TLS/X.509	Separate codec tests from a real handshake, trust path, hostname and revocation behavior.
Benchmark	Release mode, CPU features, message lengths, iterations and external load recorded.
Fuzz	Named target, seed/iterations, isolated worker results and crash triage artifacts.
Web security	Loopback binding, session bootstrap, cookie, Origin, path, body and shutdown failures.
Evidence export	JSON/HTML/advanced summary, bundle manifest, SHA-256 and ZIP extraction verified.
Cross-platform	Actual Win/MSVC and relevant Linux/macOS toolchains, socket/thread and storage integration tested.
Licensing	Authorized QRCS-CASL notices and signed/hash-identified distribution artifacts.

Table 25.1 Release qualification gates

The uploaded ZIP did not include executed validation logs or target-platform build outputs. The following API references and inventories are source inspection results, and each release gate above remains outstanding until independently executed and recorded.

26: QSCSDKC Public C Function Reference

This chapter reproduces 401 function prototypes from 43 declared project headers. For QSCSDKC these are teaching/diagnostic example entry points; for QSCSDKTest they are qualification-workbench functions. The native QSC library is external, and none of these declarations should be interpreted as the complete QSC crypto API. Parameter and return descriptions are based on adjacent header Doxygen when supplied; otherwise a purpose summary is explicitly inferred from the function name and implementation category.

26.1 QSCSDKC/acp_example.h (7 declarations)

qscsdkc_acp_run_all

qscsdkc_status qscsdkc_acp_run_all(void);

Purpose: Runs all ACP example workflows registered in this module

Return: Returns qscsdkc_status_success when all registered ACP examples pass; otherwise returns the first failing example status

qscsdkc_acp_generate

qscsdkc_status qscsdkc_acp_generate(void);

Purpose: Tests ordinary ACP byte generation

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_acp_keys

qscsdkc_status qscsdkc_acp_keys(void);

Purpose: Tests ACP key, nonce, and salt generation

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_acp_values

qscsdkc_status qscsdkc_acp_values(void);

Purpose: Tests ACP integer helper APIs

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_acp_file

qscsdkc_status qscsdkc_acp_file(void);

Purpose: Tests ACP random file generation

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_acp_limit

qscsdkc_status qscsdkc_acp_limit(void);

Purpose: Tests ACP maximum-request boundary handling

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_acp_verify

qscsdkc_status qscsdkc_acp_verify(void);

Purpose: Tests ACP repeated-output sanity checks

Return: Returns qscsdkc_status_success on success, or an error status on failure

26.2 QSCSDKC/aes_example.h (9 declarations)

qscsdkc_aes_run_all

qscsdkc_status qscsdkc_aes_run_all(void);

Purpose: Runs all AES example workflows registered in this module

Return: Returns qscsdkc_status_success when all registered AES examples pass; otherwise returns the first failing example status

qscsdkc_aes_ecb_encrypt

qscsdkc_status qscsdkc_aes_ecb_encrypt(void);

Purpose: Tests AES-256 ECB single-block encryption and decryption

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_aes_cbc_encrypt

qscsdkc_status qscsdkc_aes_cbc_encrypt(void);

Purpose: Tests AES-256 CBC encryption and decryption with PKCS#7 padding handling

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_aes_ctrbe_encrypt

qscsdkc_status qscsdkc_aes_ctrbe_encrypt(void);

Purpose: Tests AES-256 CTR big-endian counter encryption and decryption

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_aes_ctrle_encrypt

qscsdkc_status qscsdkc_aes_ctrle_encrypt(void);

Purpose: Tests AES-256 CTR little-endian counter encryption and decryption

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_aes_gcm_encrypt

qscsdkc_status qscsdkc_aes_gcm_encrypt(void);

Purpose: Tests AES-256 GCM authenticated encryption and decryption

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_aes_associated_data

qscsdkc_status qscsdkc_aes_associated_data(void);

Purpose: Tests AES-256 GCM associated-data authentication

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_aes_authentication_failure

qscsdkc_status qscsdkc_aes_authentication_failure(void);

Purpose: Tests AES-256 GCM authentication-failure handling

Return: Returns qscsdkc_status_success when tampering is detected correctly, or an error status on failure

qscsdkc_aes_file_encrypt

qscsdkc_status qscsdkc_aes_file_encrypt(void);

Purpose: Tests AES-256 GCM file encryption using an authenticated example header

Return: Returns qscsdkc_status_success on success, or an error status on failure

26.3 QSCSDKC/chacha_example.h (7 declarations)

qscsdkc_chacha_run_all

qscsdkc_status qscsdkc_chacha_run_all(void);

Purpose: Runs all ChaCha example workflows registered in this module

Return: Returns qscsdkc_status_success when all registered ChaCha examples pass; otherwise returns the first failing example status

qscsdkc_chacha_buffer_encrypt

qscsdkc_status qscsdkc_chacha_buffer_encrypt(void);

Purpose: Tests one-shot ChaCha20 encryption and decryption of a memory buffer

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_chacha_streaming_encrypt

qscsdkc_status qscsdkc_chacha_streaming_encrypt(void);

Purpose: Tests segmented ChaCha20 processing for chunked input

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_chacha_poly1305_encrypt

qscsdkc_status qscsdkc_chacha_poly1305_encrypt(void);

Purpose: Tests ChaCha20-Poly1305 authenticated encryption with associated data

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_chacha_associated_data

qscsdkc_status qscsdkc_chacha_associated_data(void);

Purpose: Tests ChaCha20-Poly1305 associated-data authentication

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_chacha_authentication_failure

qscsdkc_status qscsdkc_chacha_authentication_failure(void);

Purpose: Tests detection and handling of ChaCha20-Poly1305 authentication failure

Return: Returns qscsdkc_status_success when tampering is detected correctly, or an error status on failure

qscsdkc_chacha_file_encrypt

qscsdkc_status qscsdkc_chacha_file_encrypt(void);

Purpose: Tests ChaCha20-Poly1305 file encryption using an authenticated example header

Return: Returns qscsdkc_status_success on success, or an error status on failure

26.4 QSCSDKC/cjp_example.h (7 declarations)

qscsdkc_cjp_run_all

qscsdkc_status qscsdkc_cjp_run_all(void);

Purpose: Runs all CJP example workflows registered in this module

Return: Returns qscsdkc_status_success when all registered CJP examples pass; otherwise returns the first failing example status

qscsdkc_cjp_available

qscsdkc_status qscsdkc_cjp_available(void);

Purpose: Tests CJP provider availability

Return: Returns qscsdkc_status_success when CJP is available, or an error status on failure

qscsdkc_cjp_generate

qscsdkc_status qscsdkc_cjp_generate(void);

Purpose: Tests ordinary CJP byte generation

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_cjp_keys

qscsdkc_status qscsdkc_cjp_keys(void);

Purpose: Tests CJP key, nonce, and salt generation

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_cjp_values

qscsdkc_status qscsdkc_cjp_values(void);

Purpose: Tests CJP integer helper APIs

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_cjp_limit

qscsdkc_status qscsdkc_cjp_limit(void);

Purpose: Tests CJP maximum-request boundary handling

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_cjp_verify

qscsdkc_status qscsdkc_cjp_verify(void);

Purpose: Tests CJP repeated-output sanity checks

Return: Returns qscsdkc_status_success on success, or an error status on failure

26.5 QSCSDKC/collection_example.h (9 declarations)

qscsdkc_collection_run_all

qscsdkc_status qscsdkc_collection_run_all(void);

Purpose: Runs all registered collection and container examples

Return: Returns qscsdkc_status_success when all examples pass; otherwise returns the first failing status

qscsdkc_collection_keyed

qscsdkc_status qscsdkc_collection_keyed(void);

Purpose: Tests keyed collection add, find, and index retrieval operations

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_collection_remove

qscsdkc_status qscsdkc_collection_remove(void);

Purpose: Tests keyed collection remove and erase operations

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_collection_serialize

qscsdkc_status qscsdkc_collection_serialize(void);

Purpose: Tests keyed collection serialization and deserialization

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_collection_list

qscsdkc_status qscsdkc_collection_list(void);

Purpose: Tests list add, copy, item, count, and remove operations

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_collection_listops

qscsdkc_status qscsdkc_collection_listops(void);

Purpose: Tests list random shuffle and serialization operations

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_collection_queue

qscsdkc_status qscsdkc_collection_queue(void);

Purpose: Tests queue push, pop, flush, count, full, and empty operations

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_collection_array

qscsdkc_status qscsdkc_collection_array(void);

Purpose: Tests character-array search, parse, and hex helpers

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_collection_negative

qscsdkc_status qscsdkc_collection_negative(void);

Purpose: Tests invalid lookup, empty pop, and missing-item behavior

Return: Returns qscsdkc_status_success on success, or an error status on failure

26.6 QSCSDKC/cpuid_example.h (10 declarations)

qscsdkc_cpuid_run_all

qscsdkc_status qscsdkc_cpuid_run_all(void);

Purpose: Runs all CPUID/platform capability examples registered in this module

Return: Returns qscsdkc_status_success when all registered examples pass; otherwise returns the first failing example status

qscsdkc_cpuid_features

qscsdkc_status qscsdkc_cpuid_features(void);

Purpose: Detects and prints the primary CPU feature structure

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_cpuid_vendor

qscsdkc_status qscsdkc_cpuid_vendor(void);

Purpose: Prints vendor and CPU-type information from qsc_cpuidex_cpu_features

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_cpuid_crypto

qscsdkc_status qscsdkc_cpuid_crypto(void);

Purpose: Prints hardware cryptographic instruction availability

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_cpuid_simd

qscsdkc_status qscsdkc_cpuid_simd(void);

Purpose: Prints SIMD/vector capability availability

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_cpuid_cache

qscsdkc_status qscsdkc_cpuid_cache(void);

Purpose: Prints cache-size and cache-line information

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_cpuid_topology

qscsdkc_status qscsdkc_cpuid_topology(void);

Purpose: Prints physical-core and logical-processor topology information

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_cpuid_rdrand

qscsdkc_status qscsdkc_cpuid_rdrand(void);

Purpose: Checks hardware random-instruction availability

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_cpuid_platform

qscsdkc_status qscsdkc_cpuid_platform(void);

Purpose: Prints compile-time platform, architecture, and compiler classification

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_cpuid_print

qscsdkc_status qscsdkc_cpuid_print(void);

Purpose: Calls the QSC native CPUID statistics printer

Return: Returns qscsdkc_status_success on success, or an error status on failure

26.7 QSCSDKC/csg_example.h (10 declarations)

qscsdkc_csg_run_all

qscsdkc_status qscsdkc_csg_run_all(void);

Purpose: Runs all CSG example workflows registered in this module

Return: Returns qscsdkc_status_success when all registered CSG examples pass; otherwise returns the first failing example status

qscsdkc_csg_256

qscsdkc_status qscsdkc_csg_256(void);

Purpose: Tests CSG generation using a 256-bit seed

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_csg_512

qscsdkc_status qscsdkc_csg_512(void);

Purpose: Tests CSG generation using a 512-bit seed

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_csg_generate

qscsdkc_status qscsdkc_csg_generate(void);

Purpose: Tests ordinary CSG pseudo-random byte generation

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_csg_info

qscsdkc_status qscsdkc_csg_info(void);

Purpose: Tests CSG personalization-string domain separation

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_csg_update

qscsdkc_status qscsdkc_csg_update(void);

Purpose: Tests updating an initialized CSG state with new seed material

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_csg_stream

qscsdkc_status qscsdkc_csg_stream(void);

Purpose: Tests segmented CSG output generation

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_csg_file

qscsdkc_status qscsdkc_csg_file(void);

Purpose: Tests loading a CSG seed from a file and generating output

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_csg_verify

qscsdkc_status qscsdkc_csg_verify(void);

Purpose: Tests deterministic CSG output and personalization separation

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_csg_lifecycle

qscsdkc_status qscsdkc_csg_lifecycle(void);

Purpose: Tests the complete CSG state lifecycle

Return: Returns qscsdkc_status_success on success, or an error status on failure

26.8 QSCSDKC/csp_example.h (6 declarations)

qscsdkc_csp_run_all

qscsdkc_status qscsdkc_csp_run_all(void);

Purpose: Runs all CSP example workflows registered in this module

Return: Returns qscsdkc_status_success when all registered CSP examples pass; otherwise returns the first failing example status

qscsdkc_csp_generate

qscsdkc_status qscsdkc_csp_generate(void);

Purpose: Tests ordinary CSP byte generation

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_csp_keys

qscsdkc_status qscsdkc_csp_keys(void);

Purpose: Tests CSP key, nonce, and salt generation

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_csp_values

qscsdkc_status qscsdkc_csp_values(void);

Purpose: Tests CSP integer helper APIs

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_csp_limit

qscsdkc_status qscsdkc_csp_limit(void);

Purpose: Tests CSP maximum-request boundary handling

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_csp_verify

qscsdkc_status qscsdkc_csp_verify(void);

Purpose: Tests CSP repeated-output sanity checks

Return: Returns qscsdkc_status_success on success, or an error status on failure

26.9 QSCSDKC/csx_example.h (6 declarations)

qscsdkc_csx_run_all

qscsdkc_status qscsdkc_csx_run_all(void);

Purpose: Runs all CSX example workflows registered in this module

Return: Returns qscsdkc_status_success when all registered CSX examples pass; otherwise returns the first failing example status

qscsdkc_csx_buffer_encrypt

qscsdkc_status qscsdkc_csx_buffer_encrypt(void);

Purpose: Tests one-shot CSX-512 authenticated encryption of a memory buffer

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_csx_associated_data

qscsdkc_status qscsdkc_csx_associated_data(void);

Purpose: Tests CSX associated-data authentication

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_csx_authentication_failure

qscsdkc_status qscsdkc_csx_authentication_failure(void);

Purpose: Tests detection and handling of CSX authentication failure

Return: Returns qscsdkc_status_success when tampering is detected correctly, or an error status on failure

qscsdkc_csx_file_encrypt

qscsdkc_status qscsdkc_csx_file_encrypt(void);

Purpose: Tests CSX file encryption using an authenticated example header

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_csx_streaming_encrypt

qscsdkc_status qscsdkc_csx_streaming_encrypt(void);

Purpose: Tests segmented CSX processing for chunked input

Return: Returns qscsdkc_status_success on success, or an error status on failure

26.10 QSCSDKC/ecdh_example.h (9 declarations)

qscsdkc_ecdh_run_all

qscsdkc_status qscsdkc_ecdh_run_all(void);

Purpose: Runs all ECDH example workflows registered in this module

Return: Returns qscsdkc_status_success when all registered ECDH examples pass; otherwise returns the first failing example status

qscsdkc_ecdh_keypair

qscsdkc_status qscsdkc_ecdh_keypair(void);

Purpose: Tests ECDH public/private key-pair generation

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_ecdh_shared

qscsdkc_status qscsdkc_ecdh_shared(void);

Purpose: Tests ECDH shared-secret agreement between two peers

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_ecdh_derive

qscsdkc_status qscsdkc_ecdh_derive(void);

Purpose: Tests deriving symmetric session keys from an ECDH shared secret

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_ecdh_roles

qscsdkc_status qscsdkc_ecdh_roles(void);

Purpose: Tests client/server directional key separation from one ECDH secret

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_ecdh_publickey

qscsdkc_status qscsdkc_ecdh_publickey(void);

Purpose: Tests encoding and decoding an ECDH public key for transport

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_ecdh_file

qscsdkc_status qscsdkc_ecdh_file(void);

Purpose: Tests public-key file exchange using writable SDKC storage

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_ecdh_verify

qscsdkc_status qscsdkc_ecdh_verify(void);

Purpose: Tests deterministic ECDH consistency and session-key agreement

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_ecdh_negative

qscsdkc_status qscsdkc_ecdh_negative(void);

Purpose: Tests ECDH negative-path behavior for modified public key material

Return: Returns qscsdkc_status_success on success, or an error status on failure

26.11 QSCSDKC/ecdsa_example.h (9 declarations)

qscsdkc_ecdsa_run_all

qscsdkc_status qscsdkc_ecdsa_run_all(void);

Purpose: Runs all ECDSA example workflows registered in this module

Return: Returns qscsdkc_status_success when all registered ECDSA examples pass; otherwise returns the first failing example status

qscsdkc_ecdsa_keypair

qscsdkc_status qscsdkc_ecdsa_keypair(void);

Purpose: Tests ECDSA public/private key-pair generation

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_ecdsa_sign

qscsdkc_status qscsdkc_ecdsa_sign(void);

Purpose: Tests signing an in-memory message

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_ecdsa_verify

qscsdkc_status qscsdkc_ecdsa_verify(void);

Purpose: Tests verifying an in-memory signed message

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_ecdsa_detached

qscsdkc_status qscsdkc_ecdsa_detached(void);

Purpose: Tests detached signature creation and verification

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_ecdsa_file

qscsdkc_status qscsdkc_ecdsa_file(void);

Purpose: Tests signing and verifying file content

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_ecdsa_publickey

qscsdkc_status qscsdkc_ecdsa_publickey(void);

Purpose: Tests encoding and decoding an ECDSA public key for transport

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_ecdsa_domain

qscsdkc_status qscsdkc_ecdsa_domain(void);

Purpose: Tests application-level context-bound ECDSA signing

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_ecdsa_negative

qscsdkc_status qscsdkc_ecdsa_negative(void);

Purpose: Tests ECDSA negative-path behavior for modified messages, signatures, and keys

Return: Returns qscsdkc_status_success on success, or an error status on failure

26.12 QSCSDKC/eddh_example.h (9 declarations)

qscsdkc_eddh_run_all

qscsdkc_status qscsdkc_eddh_run_all(void);

Purpose: Runs all EDDH example workflows registered in this module

Return: Returns qscsdkc_status_success when all registered EDDH examples pass; otherwise returns the first failing example status

qscsdkc_eddh_keypair

qscsdkc_status qscsdkc_eddh_keypair(void);

Purpose: Tests EDDH public/private key-pair generation

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_eddh_shared

qscsdkc_status qscsdkc_eddh_shared(void);

Purpose: Tests EDDH shared-secret agreement between two peers

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_eddh_derive

qscsdkc_status qscsdkc_eddh_derive(void);

Purpose: Tests deriving symmetric session keys from an EDDH shared secret

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_eddh_roles

qscsdkc_status qscsdkc_eddh_roles(void);

Purpose: Tests client/server directional key separation from one EDDH secret

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_eddh_publickey

qscsdkc_status qscsdkc_eddh_publickey(void);

Purpose: Tests encoding and decoding an EDDH public key for transport

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_eddh_file

qscsdkc_status qscsdkc_eddh_file(void);

Purpose: Tests public-key file exchange using writable SDKC storage

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_eddh_verify

qscsdkc_status qscsdkc_eddh_verify(void);

Purpose: Tests deterministic EDDH consistency and session-key agreement

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_eddh_negative

qscsdkc_status qscsdkc_eddh_negative(void);

Purpose: Tests EDDH negative-path behavior for modified public key material

Return: Returns qscsdkc_status_success on success, or an error status on failure

26.13 QSCSDKC/eddsa_example.h (9 declarations)

qscsdkc_eddsa_run_all

qscsdkc_status qscsdkc_eddsa_run_all(void);

Purpose: Runs all EDDSA example workflows registered in this module

Return: Returns qscsdkc_status_success when all registered EDDSA examples pass; otherwise returns the first failing example status

qscsdkc_eddsa_keypair

qscsdkc_status qscsdkc_eddsa_keypair(void);

Purpose: Tests EDDSA public/private key-pair generation

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_eddsa_sign

qscsdkc_status qscsdkc_eddsa_sign(void);

Purpose: Tests signing an in-memory message

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_eddsa_verify

qscsdkc_status qscsdkc_eddsa_verify(void);

Purpose: Tests verifying an in-memory signed message

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_eddsa_detached

qscsdkc_status qscsdkc_eddsa_detached(void);

Purpose: Tests detached signature creation and verification

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_eddsa_file

qscsdkc_status qscsdkc_eddsa_file(void);

Purpose: Tests signing and verifying file content

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_eddsa_publickey

qscsdkc_status qscsdkc_eddsa_publickey(void);

Purpose: Tests encoding and decoding an EDDSA public key for transport

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_eddsa_domain

qscsdkc_status qscsdkc_eddsa_domain(void);

Purpose: Tests application-level context-bound EDDSA signing

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_eddsa_negative

qscsdkc_status qscsdkc_eddsa_negative(void);

Purpose: Tests EDDSA negative-path behavior for modified messages, signatures, and keys

Return: Returns qscsdkc_status_success on success, or an error status on failure

26.14 QSCSDKC/encoding_example.h (9 declarations)

qscsdkc_encoding_run_all

qscsdkc_status qscsdkc_encoding_run_all(void);

Purpose: Runs all encoding example workflows registered in this module

Return: Returns qscsdkc_status_success when all registered encoding examples pass; otherwise returns the first failing example status

qscsdkc_encoding_hex

qscsdkc_status qscsdkc_encoding_hex(void);

Purpose: Tests hexadecimal encode/decode operations

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_encoding_base64

qscsdkc_status qscsdkc_encoding_base64(void);

Purpose: Tests Base64 encode/decode operations

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_encoding_pem

qscsdkc_status qscsdkc_encoding_pem(void);

Purpose: Tests PEM wrap/unwrap operations

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_encoding_ber

qscsdkc_status qscsdkc_encoding_ber(void);

Purpose: Tests BER/DER primitive element and length encoding operations

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_encoding_roundtrip

qscsdkc_status qscsdkc_encoding_roundtrip(void);

Purpose: Tests all supported binary-to-text round-trip operations

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_encoding_sizes

qscsdkc_status qscsdkc_encoding_sizes(void);

Purpose: Tests output-size calculations for supported encodings

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_encoding_file

qscsdkc_status qscsdkc_encoding_file(void);

Purpose: Tests file encode/decode round trips using the SDKC writable storage directory

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_encoding_verify

qscsdkc_status qscsdkc_encoding_verify(void);

Purpose: Tests malformed-input rejection for supported decoders

Return: Returns qscsdkc_status_success on success, or an error status on failure

26.15 QSCSDKC/falcon_example.h (9 declarations)

qscsdkc_falcon_run_all

qscsdkc_status qscsdkc_falcon_run_all(void);

Purpose: Runs all FALCON example workflows registered in this module

Return: Returns qscsdkc_status_success when all registered FALCON examples pass; otherwise returns the first failing example status

qscsdkc_falcon_keypair

qscsdkc_status qscsdkc_falcon_keypair(void);

Purpose: Tests FALCON public/private key-pair generation

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_falcon_sign

qscsdkc_status qscsdkc_falcon_sign(void);

Purpose: Tests signing an in-memory message

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_falcon_verify

qscsdkc_status qscsdkc_falcon_verify(void);

Purpose: Tests verifying an in-memory signed message

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_falcon_detached

qscsdkc_status qscsdkc_falcon_detached(void);

Purpose: Tests detached signature creation and verification

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_falcon_file

qscsdkc_status qscsdkc_falcon_file(void);

Purpose: Tests signing and verifying file content

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_falcon_publickey

qscsdkc_status qscsdkc_falcon_publickey(void);

Purpose: Tests encoding and decoding an FALCON public key for transport

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_falcon_domain

qscsdkc_status qscsdkc_falcon_domain(void);

Purpose: Tests application-level context-bound FALCON signing

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_falcon_negative

qscsdkc_status qscsdkc_falcon_negative(void);

Purpose: Tests FALCON negative-path behavior for modified messages, signatures, and keys

Return: Returns qscsdkc_status_success on success, or an error status on failure

26.16 QSCSDKC/fileutils_example.h (10 declarations)

qscsdkc_fileutils_run_all

qscsdkc_status qscsdkc_fileutils_run_all(void);

Purpose: Runs all file and folder utility example workflows registered in this module

Return: Returns qscsdkc_status_success when all registered file and folder examples pass; otherwise returns the first failing example status

qscsdkc_fileutils_path

qscsdkc_status qscsdkc_fileutils_path(void);

Purpose: Tests writable storage-path construction and path validation

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_fileutils_write

qscsdkc_status qscsdkc_fileutils_write(void);

Purpose: Tests writing a complete byte stream to a file

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_fileutils_read

qscsdkc_status qscsdkc_fileutils_read(void);

Purpose: Tests reading a complete byte stream from a file

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_fileutils_append

qscsdkc_status qscsdkc_fileutils_append(void);

Purpose: Tests appending text data to an existing file

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_fileutils_copy

qscsdkc_status qscsdkc_fileutils_copy(void);

Purpose: Tests file copy and copied-content comparison

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_fileutils_info

qscsdkc_status qscsdkc_fileutils_info(void);

Purpose: Tests file metadata helpers such as size, name, extension, and directory

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_fileutils_folder

qscsdkc_status qscsdkc_fileutils_folder(void);

Purpose: Tests folder creation, existence checks, delimiter handling, and deletion

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_fileutils_enumerate

qscsdkc_status qscsdkc_fileutils_enumerate(void);

Purpose: Tests file and folder enumeration under the SDKC storage directory

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_fileutils_remove

qscsdkc_status qscsdkc_fileutils_remove(void);

Purpose: Tests file erase, zeroise, and delete helpers

Return: Returns qscsdkc_status_success on success, or an error status on failure

26.17 QSCSDKC/hcg_example.h (9 declarations)

qscsdkc_hcg_run_all

qscsdkc_status qscsdkc_hcg_run_all(void);

Purpose: Runs all HCG example workflows registered in this module

Return: Returns qscsdkc_status_success when all registered HCG examples pass; otherwise returns the first failing example status

qscsdkc_hcg_512

qscsdkc_status qscsdkc_hcg_512(void);

Purpose: Tests HCG generation using a 512-bit seed

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_hcg_generate

qscsdkc_status qscsdkc_hcg_generate(void);

Purpose: Tests ordinary HCG pseudo-random byte generation

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_hcg_info

qscsdkc_status qscsdkc_hcg_info(void);

Purpose: Tests HCG personalization-string domain separation

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_hcg_update

qscsdkc_status qscsdkc_hcg_update(void);

Purpose: Tests updating an initialized HCG state with new seed material

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_hcg_stream

qscsdkc_status qscsdkc_hcg_stream(void);

Purpose: Tests segmented HCG output generation

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_hcg_file

qscsdkc_status qscsdkc_hcg_file(void);

Purpose: Tests loading an HCG seed from a file and generating output

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_hcg_verify

qscsdkc_status qscsdkc_hcg_verify(void);

Purpose: Tests deterministic HCG output and personalization separation

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_hcg_lifecycle

qscsdkc_status qscsdkc_hcg_lifecycle(void);

Purpose: Tests the complete HCG state lifecycle

Return: Returns qscsdkc_status_success on success, or an error status on failure

26.18 QSCSDKC/hkdf_example.h (9 declarations)

qscsdkc_hkdf_run_all

qscsdkc_status qscsdkc_hkdf_run_all(void);

Purpose: Runs all HKDF example workflows registered in this module

Return: Returns qscsdkc_status_success when all registered HKDF examples pass; otherwise returns the first failing example status

qscsdkc_hkdf_256

qscsdkc_status qscsdkc_hkdf_256(void);

Purpose: Tests a SHA2-HKDF-256 extract-and-expand derivation

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_hkdf_512

qscsdkc_status qscsdkc_hkdf_512(void);

Purpose: Tests a SHA2-HKDF-512 extract-and-expand derivation

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_hkdf_derive

qscsdkc_status qscsdkc_hkdf_derive(void);

Purpose: Tests direct key derivation for HKDF-256 and HKDF-512

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_hkdf_extract

qscsdkc_status qscsdkc_hkdf_extract(void);

Purpose: Tests the explicit HKDF extract and expand workflow

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_hkdf_keys

qscsdkc_status qscsdkc_hkdf_keys(void);

Purpose: Tests deriving multiple independent protocol keys with unique info labels

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_hkdf_info

qscsdkc_status qscsdkc_hkdf_info(void);

Purpose: Tests HKDF info-field domain separation

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_hkdf_verify

qscsdkc_status qscsdkc_hkdf_verify(void);

Purpose: Tests deterministic HKDF output and changed-info rejection

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_hkdf_file

qscsdkc_status qscsdkc_hkdf_file(void);

Purpose: Tests deriving HKDF output from a demonstration key file

Return: Returns qscsdkc_status_success on success, or an error status on failure

26.19 QSCSDKC/hmac_example.h (6 declarations)

qscsdkc_hmac_run_all

qscsdkc_status qscsdkc_hmac_run_all(void);

Purpose: Runs all HMAC example workflows registered in this module

Return: Returns qscsdkc_status_success when all registered HMAC examples pass; otherwise returns the first failing example status

qscsdkc_hmac_256

qscsdkc_status qscsdkc_hmac_256(void);

Purpose: Tests HMAC(SHA2-256) one-shot and streaming MAC processing

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_hmac_512

qscsdkc_status qscsdkc_hmac_512(void);

Purpose: Tests HMAC(SHA2-512) one-shot and streaming MAC processing

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_hmac_stream

qscsdkc_status qscsdkc_hmac_stream(void);

Purpose: Tests HMAC streaming examples for HMAC-256 and HMAC-512

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_hmac_file

qscsdkc_status qscsdkc_hmac_file(void);

Purpose: Tests HMAC file examples for HMAC-256 and HMAC-512

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_hmac_verify

qscsdkc_status qscsdkc_hmac_verify(void);

Purpose: Tests HMAC verification and modified-message rejection

Return: Returns qscsdkc_status_success on success, or an error status on failure

26.20 QSCSDKC/hqc_example.h (10 declarations)

qscsdkc_hqc_run_all

qscsdkc_status qscsdkc_hqc_run_all(void);

Purpose: Runs all HQC example workflows registered in this module

Return: Returns qscsdkc_status_success when all registered HQC examples pass; otherwise returns the first failing example status

qscsdkc_hqc_keypair

qscsdkc_status qscsdkc_hqc_keypair(void);

Purpose: Tests HQC public/private key-pair generation

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_hqc_encapsulate

qscsdkc_status qscsdkc_hqc_encapsulate(void);

Purpose: Tests HQC encapsulation against a generated public key

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_hqc_decapsulate

qscsdkc_status qscsdkc_hqc_decapsulate(void);

Purpose: Tests HQC decapsulation against a matching private key and ciphertext

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_hqc_derive

qscsdkc_status qscsdkc_hqc_derive(void);

Purpose: Tests deriving symmetric session keys from an HQC shared secret

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_hqc_roles

qscsdkc_status qscsdkc_hqc_roles(void);

Purpose: Tests client/server directional key separation from one HQC secret

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_hqc_publickey

qscsdkc_status qscsdkc_hqc_publickey(void);

Purpose: Tests encoding and decoding an HQC public key for transport

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_hqc_file

qscsdkc_status qscsdkc_hqc_file(void);

Purpose: Tests public-key file exchange using writable SDKC storage

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_hqc_verify

qscsdkc_status qscsdkc_hqc_verify(void);

Purpose: Tests HQC encapsulation and decapsulation agreement

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_hqc_negative

qscsdkc_status qscsdkc_hqc_negative(void);

Purpose: Tests HQC negative-path behavior for modified ciphertext material

Return: Returns qscsdkc_status_success on success, or an error status on failure

26.21 QSCSDKC/intutils_example.h (11 declarations)

qscsdkc_intutils_run_all

qscsdkc_status qscsdkc_intutils_run_all(void);

Purpose: Runs all integer utility example workflows registered in this module

Return: Returns qscsdkc_status_success when all registered integer utility examples pass; otherwise returns the first failing example status

qscsdkc_intutils_be

qscsdkc_status qscsdkc_intutils_be(void);

Purpose: Tests big-endian integer serialization and parsing helpers

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_intutils_le

qscsdkc_status qscsdkc_intutils_le(void);

Purpose: Tests little-endian integer serialization and parsing helpers

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_intutils_increment

qscsdkc_status qscsdkc_intutils_increment(void);

Purpose: Tests big-endian and little-endian byte-array counter increments

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_intutils_rotate

qscsdkc_status qscsdkc_intutils_rotate(void);

Purpose: Tests 32-bit and 64-bit rotate helpers

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_intutils_bit

qscsdkc_status qscsdkc_intutils_bit(void);

Purpose: Tests bit-reversal, bit-count, and bit-state helpers

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_intutils_verify

qscsdkc_status qscsdkc_intutils_verify(void);

Purpose: Tests constant-time verification and conditional move helpers

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_intutils_mask

qscsdkc_status qscsdkc_intutils_mask(void);

Purpose: Tests size comparison, min/max, power-of-two, and mask helpers

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_intutils_hex

qscsdkc_status qscsdkc_intutils_hex(void);

Purpose: Tests hex-to-binary and binary-to-hex conversion helpers

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_intutils_clear

qscsdkc_status qscsdkc_intutils_clear(void);

Purpose: Tests typed integer-array clear helpers

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_intutils_math

qscsdkc_status qscsdkc_intutils_math(void);

Purpose: Tests selected integer math helper functions

Return: Returns qscsdkc_status_success on success, or an error status on failure

26.22 QSCSDKC/kmac_example.h (6 declarations)

qscsdkc_kmac_run_all

qscsdkc_status qscsdkc_kmac_run_all(void);

Purpose: Runs all KMAC example workflows registered in this module

Return: Returns qscsdkc_status_success when all registered KMAC examples pass; otherwise returns the first failing example status

qscsdkc_kmac_256

qscsdkc_status qscsdkc_kmac_256(void);

Purpose: Tests KMAC-256 one-shot and streaming MAC processing

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_kmac_512

qscsdkc_status qscsdkc_kmac_512(void);

Purpose: Tests KMAC-512 one-shot and streaming MAC processing

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_kmac_stream

qscsdkc_status qscsdkc_kmac_stream(void);

Purpose: Tests KMAC streaming examples for KMAC-256 and KMAC-512

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_kmac_file

qscsdkc_status qscsdkc_kmac_file(void);

Purpose: Tests KMAC file examples for KMAC-256 and KMAC-512

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_kmac_verify

qscsdkc_status qscsdkc_kmac_verify(void);

Purpose: Tests KMAC verification and modified-message rejection

Return: Returns qscsdkc_status_success on success, or an error status on failure

26.23 QSCSDKC/mceliece_example.h (10 declarations)

qscsdkc_mceliece_run_all

qscsdkc_status qscsdkc_mceliece_run_all(void);

Purpose: Runs all Classic McEliece example tests. Executes the complete McEliece example group, including key-pair generation, encapsulation, decapsulation, shared-secret verification, HKDF-based key derivation, public-key encoding, file-based public-key exchange, and negative ciphertext rejection

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating the first failed McEliece example

qscsdkc_mceliece_keypair

qscsdkc_status qscsdkc_mceliece_keypair(void);

Purpose: Runs the Classic McEliece key-pair generation example. Demonstrates how to generate a McEliece public/private key pair using the QSC McEliece API. The public key is intended for distribution to an encapsulating peer, while the private key must remain secret and is cleared from temporary buffers before the example returns

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating key-pair generation failure

qscsdkc_mceliece_encapsulate

qscsdkc_status qscsdkc_mceliece_encapsulate(void);

Purpose: Runs the Classic McEliece encapsulation example. Demonstrates how an initiator uses a peer McEliece public key to generate a ciphertext and a shared secret. The ciphertext is transmitted to the private key holder, while the shared secret is used as input key material for a KDF

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating encapsulation failure

qscsdkc_mceliece_decapsulate

qscsdkc_status qscsdkc_mceliece_decapsulate(void);

Purpose: Runs the Classic McEliece decapsulation example. Demonstrates how the private key holder decapsulates a McEliece ciphertext to recover the shared secret generated by the encapsulating peer. The example verifies that decapsulation succeeds for an authentic ciphertext

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating decapsulation failure

qscsdkc_mceliece_derive

qscsdkc_status qscsdkc_mceliece_derive(void);

Purpose: Runs the McEliece shared-secret key-derivation example. Demonstrates the recommended application pattern of passing the raw McEliece shared secret through HKDF before using it as application key material. The example derives encryption, authentication, or nonce-seed material from the shared secret using explicit domain-separation labels

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating encapsulation, decapsulation, or derivation failure

qscsdkc_mceliece_roles

qscsdkc_status qscsdkc_mceliece_roles(void);

Purpose: Runs the McEliece client/server role-separation example. Demonstrates how two peers can derive directional application keys from the same McEliece shared secret. The example uses distinct HKDF info labels for client-to-server and server-to-client material so that the same derived key is not reused in both traffic directions

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating role-based derivation failure

qscsdkc_mceliece_publickey

qscsdkc_status qscsdkc_mceliece_publickey(void);

Purpose: Runs the McEliece public-key encoding example. Demonstrates how to encode and decode a McEliece public key for transport or storage using QSC encoding utilities. Classic McEliece public keys are large, so the example uses heap-backed buffers rather than large stack allocations

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating public-key encoding or round-trip verification failure

qscsdkc_mceliece_file

qscsdkc_status qscsdkc_mceliece_file(void);

Purpose: Runs the McEliece public-key file exchange example. Demonstrates a simple file-based public-key exchange workflow. The example writes public keys to files in the SDKC storage directory, reads them back, and uses the recovered public keys for McEliece encapsulation. Private keys are not written by this example

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating file I/O, encoding, or shared-secret verification failure

qscsdkc_mceliece_verify

qscsdkc_status qscsdkc_mceliece_verify(void);

Purpose: Runs the McEliece encapsulation/decapsulation verification example. Generates a McEliece key pair, encapsulates a shared secret with the public key, decapsulates the ciphertext with the private key, and verifies that both peers derive the same shared secret

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating shared-secret mismatch or KEM operation failure

qscsdkc_mceliece_negative

qscsdkc_status qscsdkc_mceliece_negative(void);

Purpose: Runs the McEliece negative-test example. Demonstrates secure rejection behavior by modifying a McEliece ciphertext and confirming that the decapsulated result does not authenticate as the original shared secret. Applications must treat modified or invalid ciphertexts as failed key-establishment attempts

Return: Returns qscsdkc_status_success if the negative test is rejected as expected; otherwise, returns an error status indicating unexpected acceptance or verification failure

26.24 QSCSDKC/memutils_example.h (13 declarations)

qscsdkc_memutils_run_all

qscsdkc_status qscsdkc_memutils_run_all(void);

Purpose: Runs all memory utility example workflows registered in this module

Return: Returns qscsdkc_status_success when all registered memory utility examples pass; otherwise returns the first failing example status

qscsdkc_memutils_copy

qscsdkc_status qscsdkc_memutils_copy(void);

Purpose: Tests qsc_memutils_copy with caller-owned buffers

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_memutils_move

qscsdkc_status qscsdkc_memutils_move(void);

Purpose: Tests qsc_memutils_move with overlapping buffers

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_memutils_clear

qscsdkc_status qscsdkc_memutils_clear(void);

Purpose: Tests qsc_memutils_clear and qsc_memutils_zeroed

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_memutils_secureerase

qscsdkc_status qscsdkc_memutils_secureerase(void);

Purpose: Tests qsc_memutils_secure_erase

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_memutils_compare

qscsdkc_status qscsdkc_memutils_compare(void);

Purpose: Tests byte-array equality and fixed-width equality helpers

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_memutils_uniform

qscsdkc_status qscsdkc_memutils_uniform(void);

Purpose: Tests qsc_memutils_array_uniform

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_memutils_xor

qscsdkc_status qscsdkc_memutils_xor(void);

Purpose: Tests qsc_memutils_xor and qsc_memutils_xorv

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_memutils_alloc

qscsdkc_status qscsdkc_memutils_alloc(void);

Purpose: Tests qsc_memutils_malloc, qsc_memutils_realloc, and qsc_memutils_alloc_free

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_memutils_aligned

qscsdkc_status qscsdkc_memutils_aligned(void);

Purpose: Tests qsc_memutils_aligned_alloc, qsc_memutils_aligned_realloc, and aligned free

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_memutils_secure

qscsdkc_status qscsdkc_memutils_secure(void);

Purpose: Tests qsc_memutils_secure_malloc and qsc_memutils_secure_free

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_memutils_zeroed

qscsdkc_status qscsdkc_memutils_zeroed(void);

Purpose: Tests qsc_memutils_zeroed directly

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_memutils_page

qscsdkc_status qscsdkc_memutils_page(void);

Purpose: Tests memory page, prefetch, and cache-line utility calls

Return: Returns qscsdkc_status_success on success, or an error status on failure

26.25 QSCSDKC/mldsa_example.h (9 declarations)

qscsdkc_mldsa_run_all

qscsdkc_status qscsdkc_mldsa_run_all(void);

Purpose: Runs all ML-DSA example workflows registered in this module

Return: Returns qscsdkc_status_success when all registered ML-DSA examples pass; otherwise returns the first failing example status

qscsdkc_mldsa_keypair

qscsdkc_status qscsdkc_mldsa_keypair(void);

Purpose: Tests ML-DSA public/private key-pair generation

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_mldsa_sign

qscsdkc_status qscsdkc_mldsa_sign(void);

Purpose: Tests signing an in-memory message

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_mldsa_verify

qscsdkc_status qscsdkc_mldsa_verify(void);

Purpose: Tests verifying an in-memory signed message

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_mldsa_detached

qscsdkc_status qscsdkc_mldsa_detached(void);

Purpose: Tests detached signature creation and verification

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_mldsa_file

qscsdkc_status qscsdkc_mldsa_file(void);

Purpose: Tests signing and verifying file content

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_mldsa_publickey

qscsdkc_status qscsdkc_mldsa_publickey(void);

Purpose: Tests encoding and decoding an ML-DSA public key for transport

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_mldsa_domain

qscsdkc_status qscsdkc_mldsa_domain(void);

Purpose: Tests context-bound/domain-separated ML-DSA signing

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_mldsa_negative

qscsdkc_status qscsdkc_mldsa_negative(void);

Purpose: Tests ML-DSA negative-path behavior for modified messages, signatures, and keys

Return: Returns qscsdkc_status_success on success, or an error status on failure

26.26 QSCSDKC/mlkem_example.h (10 declarations)

qscsdkc_mlkem_run_all

qscsdkc_status qscsdkc_mlkem_run_all(void);

Purpose: Runs all ML-KEM example workflows registered in this module

Return: Returns qscsdkc_status_success when all registered ML-KEM examples pass; otherwise returns the first failing example status

qscsdkc_mlkem_keypair

qscsdkc_status qscsdkc_mlkem_keypair(void);

Purpose: Tests ML-KEM public/private key-pair generation

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_mlkem_encapsulate

qscsdkc_status qscsdkc_mlkem_encapsulate(void);

Purpose: Tests ML-KEM encapsulation against a generated public key

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_mlkem_decapsulate

qscsdkc_status qscsdkc_mlkem_decapsulate(void);

Purpose: Tests ML-KEM decapsulation against a matching private key and ciphertext

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_mlkem_derive

qscsdkc_status qscsdkc_mlkem_derive(void);

Purpose: Tests deriving symmetric session keys from an ML-KEM shared secret

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_mlkem_roles

qscsdkc_status qscsdkc_mlkem_roles(void);

Purpose: Tests client/server directional key separation from one ML-KEM secret

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_mlkem_publickey

qscsdkc_status qscsdkc_mlkem_publickey(void);

Purpose: Tests encoding and decoding an ML-KEM public key for transport

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_mlkem_file

qscsdkc_status qscsdkc_mlkem_file(void);

Purpose: Tests public-key file exchange using writable SDKC storage

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_mlkem_verify

qscsdkc_status qscsdkc_mlkem_verify(void);

Purpose: Tests ML-KEM encapsulation and decapsulation agreement

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_mlkem_negative

qscsdkc_status qscsdkc_mlkem_negative(void);

Purpose: Tests ML-KEM negative-path behavior for modified ciphertext material

Return: Returns qscsdkc_status_success on success, or an error status on failure

26.27 QSCSDKC/pbmac1_example.h (8 declarations)

qscsdkc_pbmac1_run_all

qscsdkc_status qscsdkc_pbmac1_run_all(void);

Purpose: Runs all PBMAC1 example workflows registered in this module

Return: Returns qscsdkc_status_success when all registered PBMAC1 examples pass; otherwise returns the first failing example status

qscsdkc_pbmac1_256

qscsdkc_status qscsdkc_pbmac1_256(void);

Purpose: Tests PBMAC1 using PBKDF2-HMAC-SHA2-256 and HMAC-SHA2-256

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_pbmac1_384

qscsdkc_status qscsdkc_pbmac1_384(void);

Purpose: Tests PBMAC1 using PBKDF2-HMAC-SHA2-384 and HMAC-SHA2-384

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_pbmac1_512

qscsdkc_status qscsdkc_pbmac1_512(void);

Purpose: Tests PBMAC1 using PBKDF2-HMAC-SHA2-512 and HMAC-SHA2-512

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_pbmac1_derive

qscsdkc_status qscsdkc_pbmac1_derive(void);

Purpose: Tests PBMAC1 derived-key generation with PBKDF2

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_pbmac1_stream

qscsdkc_status qscsdkc_pbmac1_stream(void);

Purpose: Tests PBMAC1 streaming MAC processing

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_pbmac1_file

qscsdkc_status qscsdkc_pbmac1_file(void);

Purpose: Tests PBMAC1 file MAC processing

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_pbmac1_verify

qscsdkc_status qscsdkc_pbmac1_verify(void);

Purpose: Tests PBMAC1 verification and modified-message rejection

Return: Returns qscsdkc_status_success on success, or an error status on failure

26.28 QSCSDKC/qmac_example.h (5 declarations)

qscsdkc_qmac_run_all

qscsdkc_status qscsdkc_qmac_run_all(void);

Purpose: Runs all QMAC example workflows registered in this module

Return: Returns qscsdkc_status_success when all registered QMAC examples pass; otherwise returns the first failing example status

qscsdkc_qmac_compute

qscsdkc_status qscsdkc_qmac_compute(void);

Purpose: Tests QMAC one-shot MAC processing

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_qmac_stream

qscsdkc_status qscsdkc_qmac_stream(void);

Purpose: Tests QMAC streaming MAC processing

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_qmac_file

qscsdkc_status qscsdkc_qmac_file(void);

Purpose: Tests QMAC file MAC processing

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_qmac_verify

qscsdkc_status qscsdkc_qmac_verify(void);

Purpose: Tests QMAC verification and modified-message rejection

Return: Returns qscsdkc_status_success on success, or an error status on failure

26.29 QSCSDKC/rcs_example.h (6 declarations)

qscsdkc_rcs_run_all

qscsdkc_status qscsdkc_rcs_run_all(void);

Purpose: Runs all RCS example workflows registered in this module

Return: Returns qscsdkc_status_success when all registered RCS examples pass; otherwise returns the first failing example status

qscsdkc_rcs_buffer_encrypt

qscsdkc_status qscsdkc_rcs_buffer_encrypt(void);

Purpose: Tests one-shot RCS-256 authenticated encryption of a memory buffer

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_rcs_associated_data

qscsdkc_status qscsdkc_rcs_associated_data(void);

Purpose: Tests RCS associated-data authentication

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_rcs_authentication_failure

qscsdkc_status qscsdkc_rcs_authentication_failure(void);

Purpose: Tests detection and handling of RCS authentication failure

Return: Returns qscsdkc_status_success when tampering is detected correctly, or an error status on failure

qscsdkc_rcs_file_encrypt

qscsdkc_status qscsdkc_rcs_file_encrypt(void);

Purpose: Tests RCS file encryption using an authenticated example header

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_rcs_streaming_encrypt

qscsdkc_status qscsdkc_rcs_streaming_encrypt(void);

Purpose: Tests segmented RCS processing for chunked input

Return: Returns qscsdkc_status_success on success, or an error status on failure

26.30 QSCSDKC/rdp_example.h (7 declarations)

qscsdkc_rdp_run_all

qscsdkc_status qscsdkc_rdp_run_all(void);

Purpose: Runs all RDP example workflows registered in this module

Return: Returns qscsdkc_status_success when all registered RDP examples pass; otherwise returns the first failing example status

qscsdkc_rdp_available

qscsdkc_status qscsdkc_rdp_available(void);

Purpose: Tests whether the RDRAND entropy provider is available

Return: Returns qscsdkc_status_success when RDRAND is available, or an error status otherwise

qscsdkc_rdp_generate

qscsdkc_status qscsdkc_rdp_generate(void);

Purpose: Tests ordinary RDP byte generation

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_rdp_keys

qscsdkc_status qscsdkc_rdp_keys(void);

Purpose: Tests RDP key, nonce, and salt generation

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_rdp_values

qscsdkc_status qscsdkc_rdp_values(void);

Purpose: Tests RDP integer helper APIs

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_rdp_limit

qscsdkc_status qscsdkc_rdp_limit(void);

Purpose: Tests RDP maximum-request boundary handling

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_rdp_verify

qscsdkc_status qscsdkc_rdp_verify(void);

Purpose: Tests RDP repeated-output sanity checks

Return: Returns qscsdkc_status_success on success, or an error status on failure

26.31 QSCSDKC/scb_example.h (9 declarations)

qscsdkc_scb_run_all

qscsdkc_status qscsdkc_scb_run_all(void);

Purpose: Runs all SCB example workflows registered in this module

Return: Returns qscsdkc_status_success when all registered SCB examples pass; otherwise returns the first failing example status

qscsdkc_scb_256

qscsdkc_status qscsdkc_scb_256(void);

Purpose: Tests the SCB-256 known-answer output path

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_scb_512

qscsdkc_status qscsdkc_scb_512(void);

Purpose: Tests the SCB-512 known-answer output path

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_scb_generate

qscsdkc_status qscsdkc_scb_generate(void);

Purpose: Tests generating SCB output from generated seed material

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_scb_info

qscsdkc_status qscsdkc_scb_info(void);

Purpose: Tests SCB personalization-string domain separation

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_scb_update

qscsdkc_status qscsdkc_scb_update(void);

Purpose: Tests updating an initialized SCB state with new seed material

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_scb_stream

qscsdkc_status qscsdkc_scb_stream(void);

Purpose: Tests segmented SCB output generation

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_scb_file

qscsdkc_status qscsdkc_scb_file(void);

Purpose: Tests loading an SCB seed from a file and generating output

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_scb_verify

qscsdkc_status qscsdkc_scb_verify(void);

Purpose: Tests deterministic SCB output and changed-info rejection

Return: Returns qscsdkc_status_success on success, or an error status on failure

26.32 QSCSDKC/sdkcutils.h (10 declarations)

qscsdkc_command_equals

bool qscsdkc_command_equals(const char* arg, const char* token);

Purpose: Compares a command-line argument to an expected switch

Parameters: arg: The command-line argument.; token: The expected switch token.

Return: Returns true when the argument and token are equal; otherwise false

qscsdkc_print_banner

void qscsdkc_print_banner(const char* title);

Purpose: Prints a top-level example banner

Parameters: title: The banner title.

qscsdkc_print_section

void qscsdkc_print_section(const char* title);

Purpose: Prints a named section heading

Parameters: title: The section title.

qscsdkc_print_line

void qscsdkc_print_line(const char* text);

Purpose: Prints a single console line

Parameters: text: The text to print.

qscsdkc_print_blank

void qscsdkc_print_blank(void);

Purpose: Prints a blank console line

qscsdkc_print_indent

void qscsdkc_print_indent(const char* text);

Purpose: Prints an indented console line

Parameters: text: The text to print.

qscsdkc_print_result

void qscsdkc_print_result(const char* label, qscsdkc_status status);

Purpose: Prints a named result with its status string

Parameters: label: The result label.; status: The result status.

qscsdkc_get_default_storage_path

bool qscsdkc_get_default_storage_path(char* fpath, size_t pathlen);

Purpose: Gets the default writable storage directory used by SDK examples

Parameters: fpath: The output path buffer.; pathlen: The output path buffer length.

Return: Returns true when the directory exists or was created

qscsdkc_get_storage_file_path

bool qscsdkc_get_storage_file_path(char* fpath, size_t pathlen, const char* fname);

Purpose: Builds a writable file path inside the SDK example storage directory

Parameters: fpath: The output path buffer.; pathlen: The output path buffer length.; fname: The file name to append to the storage directory.

Return: Returns true when the path was created successfully

qscsdkc_status_to_string

const char* qscsdkc_status_to_string(qscsdkc_status status);

Purpose: Converts an example status code to a display string

Parameters: status: The status code to convert.

Return: Returns a constant string describing the status code

26.33 QSCSDKC/secrand_example.h (11 declarations)

qscsdkc_secrand_run_all

qscsdkc_status qscsdkc_secrand_run_all(void);

Purpose: Runs all secrand example workflows registered in this module

Return: Returns qscsdkc_status_success when all registered secrand examples pass; otherwise returns the first failing example status

qscsdkc_secrand_256

qscsdkc_status qscsdkc_secrand_256(void);

Purpose: Tests secrand generation using a 256-bit seed

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_secrand_generate

qscsdkc_status qscsdkc_secrand_generate(void);

Purpose: Tests ordinary secrand byte generation

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_secrand_keys

qscsdkc_status qscsdkc_secrand_keys(void);

Purpose: Tests secrand key, nonce, and salt material generation

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_secrand_values

qscsdkc_status qscsdkc_secrand_values(void);

Purpose: Tests secrand integer and floating-point helper APIs

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_secrand_bounded

qscsdkc_status qscsdkc_secrand_bounded(void);

Purpose: Tests secrand bounded integer helper APIs

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_secrand_info

qscsdkc_status qscsdkc_secrand_info(void);

Purpose: Tests secrand personalization-string domain separation

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_secrand_stream

qscsdkc_status qscsdkc_secrand_stream(void);

Purpose: Tests segmented secrand output generation

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_secrand_file

qscsdkc_status qscsdkc_secrand_file(void);

Purpose: Tests loading a secrand seed from a file and generating output

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_secrand_verify

qscsdkc_status qscsdkc_secrand_verify(void);

Purpose: Tests deterministic secrand output and personalization separation

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_secrand_lifecycle

qscsdkc_status qscsdkc_secrand_lifecycle(void);

Purpose: Tests the complete secrand state lifecycle

Return: Returns qscsdkc_status_success on success, or an error status on failure

26.34 QSCSDKC/sha2_example.h (8 declarations)

qscsdkc_sha2_run_all

qscsdkc_status qscsdkc_sha2_run_all(void);

Purpose: Runs all SHA2 example workflows registered in this module

Return: Returns qscsdkc_status_success when all registered SHA2 examples pass; otherwise returns the first failing example status

qscsdkc_sha2_256

qscsdkc_status qscsdkc_sha2_256(void);

Purpose: Tests SHA2-256 short-form and long-form digest processing

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_sha2_384

qscsdkc_status qscsdkc_sha2_384(void);

Purpose: Tests SHA2-384 short-form and long-form digest processing

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_sha2_512

qscsdkc_status qscsdkc_sha2_512(void);

Purpose: Tests SHA2-512 short-form and long-form digest processing

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_sha2_short

qscsdkc_status qscsdkc_sha2_short(void);

Purpose: Tests SHA2 short-message digest examples for all supported digest widths

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_sha2_long

qscsdkc_status qscsdkc_sha2_long(void);

Purpose: Tests SHA2 long-message digest examples for all supported digest widths

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_sha2_stream

qscsdkc_status qscsdkc_sha2_stream(void);

Purpose: Tests SHA2 streaming digest examples for all supported digest widths

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_sha2_file

qscsdkc_status qscsdkc_sha2_file(void);

Purpose: Tests SHA2 file digest examples for all supported digest widths

Return: Returns qscsdkc_status_success on success, or an error status on failure

26.35 QSCSDKC/sha3_example.h (8 declarations)

qscsdkc_sha3_run_all

qscsdkc_status qscsdkc_sha3_run_all(void);

Purpose: Runs all SHA3 example workflows registered in this module

Return: Returns qscsdkc_status_success when all registered SHA3 examples pass; otherwise returns the first failing example status

qscsdkc_sha3_128

qscsdkc_status qscsdkc_sha3_128(void);

Purpose: Tests SHA3-128 short-form and long-form digest processing

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_sha3_256

qscsdkc_status qscsdkc_sha3_256(void);

Purpose: Tests SHA3-256 short-form and long-form digest processing

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_sha3_512

qscsdkc_status qscsdkc_sha3_512(void);

Purpose: Tests SHA3-512 short-form and long-form digest processing

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_sha3_short

qscsdkc_status qscsdkc_sha3_short(void);

Purpose: Tests SHA3 short-message digest examples for all supported digest widths

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_sha3_long

qscsdkc_status qscsdkc_sha3_long(void);

Purpose: Tests SHA3 long-message digest examples for all supported digest widths

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_sha3_stream

qscsdkc_status qscsdkc_sha3_stream(void);

Purpose: Tests SHA3 streaming digest examples for all supported digest widths

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_sha3_file

qscsdkc_status qscsdkc_sha3_file(void);

Purpose: Tests SHA3 file digest examples for all supported digest widths

Return: Returns qscsdkc_status_success on success, or an error status on failure

26.36 QSCSDKC/shake_example.h (17 declarations)

qscsdkc_shake_run_all

qscsdkc_status qscsdkc_shake_run_all(void);

Purpose: Runs all SHAKE example tests. Executes the complete SHAKE example group, including SHAKE-128, SHAKE-256, SHAKE-512, fixed-length output generation, streaming output, labeled key-material generation, file-derived output, and deterministic output verification

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating the first failed SHAKE example

qscsdkc_shake_128

qscsdkc_status qscsdkc_shake_128(void);

Purpose: Runs the SHAKE-128 output-generation example. Demonstrates SHAKE-128 as an extendable-output function by absorbing an input message and producing caller-sized output. The example shows that SHAKE output length is selected by the application rather than fixed by the primitive

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating SHAKE-128 generation or verification failure

qscsdkc_shake_256

qscsdkc_status qscsdkc_shake_256(void);

Purpose: Runs the SHAKE-256 output-generation example. Demonstrates SHAKE-256 as an extendable-output function suitable for generating deterministic output from an input message. The example validates buffer sizing and compares repeated derivations for deterministic equality

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating SHAKE-256 generation or verification failure

qscsdkc_shake_512

qscsdkc_status qscsdkc_shake_512(void);

Purpose: Runs the SHAKE-512 output-generation example. Demonstrates SHAKE-512 as an extendable-output function for applications that require the larger SHAKE security-strength variant exposed by QSC. The example generates fixed-size output and verifies deterministic reproduction from the same input

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating SHAKE-512 generation or verification failure

qscsdkc_shake_generate

qscsdkc_status qscsdkc_shake_generate(void);

Purpose: Runs the SHAKE fixed-output generation example. Demonstrates how to generate an arbitrary-length output buffer from input key material or message data using SHAKE. This example is the basic copy-paste pattern for applications that need deterministic expansion

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating output generation failure

qscsdkc_shake_stream

qscsdkc_status qscsdkc_shake_stream(void);

Purpose: Runs the SHAKE streaming example. Demonstrates the incremental SHAKE workflow, including state initialization, segmented input absorption, output squeezing, and state cleanup. This pattern is useful when input data is not available in a single contiguous buffer

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating streaming generation or output verification failure

qscsdkc_shake_keys

qscsdkc_status qscsdkc_shake_keys(void);

Purpose: Runs the SHAKE key-material derivation example. Demonstrates how to use SHAKE output as deterministic key material for multiple application purposes. The example derives separate labeled

buffers such as encryption-key material, authentication-key material, and nonce-seed material, and documents that applications must keep purpose separation clear

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating key-material generation or separation failure

qscsdkc_shake_file

qscsdkc_status qscsdkc_shake_file(void);

Purpose: Runs the SHAKE file-derived output example. Demonstrates deriving SHAKE output from file content read through QSC file utilities. The example treats the file as input data for deterministic expansion and verifies that repeated processing of the same file produces the same output

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating file I/O, SHAKE generation, or verification failure

qscsdkc_shake_verify

qscsdkc_status qscsdkc_shake_verify(void);

Purpose: Runs the SHAKE deterministic-verification example. Demonstrates that SHAKE produces the same output for the same input and output length, and different output when the input or requested output size changes. This example is useful for test fixtures and reproducible derivation checks

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating deterministic verification failure

qscsdkc_cshake_run_all

qscsdkc_status qscsdkc_cshake_run_all(void);

Purpose: Runs all cSHAKE example tests. Executes the complete cSHAKE example group, including cSHAKE-128, cSHAKE-256, cSHAKE-512, domain-separated output generation, labeled key-material generation, file-derived output, and deterministic customization verification

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating the first failed cSHAKE example

qscsdkc_cshake_128

qscsdkc_status qscsdkc_cshake_128(void);

Purpose: Runs the cSHAKE-128 customization example. Demonstrates cSHAKE-128 output generation using function-name and customization inputs. The example shows how cSHAKE extends SHAKE with explicit domain-separation fields

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating cSHAKE-128 generation or verification failure

qscsdkc_cshake_256

qscsdkc_status qscsdkc_cshake_256(void);

Purpose: Runs the cSHAKE-256 customization example. Demonstrates cSHAKE-256 output generation with explicit customization data. The example verifies that the same input and customization reproduce the same output and that different customization strings produce different outputs

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating cSHAKE-256 generation or verification failure

qscsdkc_cshake_512

qscsdkc_status qscsdkc_cshake_512(void);

Purpose: Runs the cSHAKE-512 customization example. Demonstrates cSHAKE-512 output generation for applications that require the larger cSHAKE security-strength variant exposed by QSC. The example uses explicit customization input and verifies deterministic reproduction

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating cSHAKE-512 generation or verification failure

qscsdkc_cshake_domain

qscsdkc_status qscsdkc_cshake_domain(void);

Purpose: Runs the cSHAKE domain-separation example. Demonstrates how cSHAKE customization strings separate application domains. The example derives output from the same input under two different customization labels and verifies that the derived outputs differ

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating domain-separation failure

qscsdkc_cshake_keys

qscsdkc_status qscsdkc_cshake_keys(void);

Purpose: Runs the cSHAKE key-material derivation example. Demonstrates using cSHAKE to derive purpose-specific key material with explicit customization labels. The example derives separate buffers for encryption, authentication, and nonce-seed material using distinct labels

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating key-material derivation or label separation failure

qscsdkc_cshake_file

qscsdkc_status qscsdkc_cshake_file(void);

Purpose: Runs the cSHAKE file-derived output example. Demonstrates deriving cSHAKE output from file content while binding the derivation to a customization string. The example verifies deterministic output for the same file and customization input

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating file I/O, cSHAKE generation, or verification failure

qscsdkc_cshake_verify

qscsdkc_status qscsdkc_cshake_verify(void);

Purpose: Runs the cSHAKE deterministic customization-verification example. Demonstrates that cSHAKE reproduces identical output for the same input, function-name string, customization string, and output length, while changed customization data produces distinct output

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating deterministic verification or customization separation failure

26.37 QSCSDKC/socketutils_example.h (10 declarations)

qscsdkc_socketutils_run_all

qscsdkc_status qscsdkc_socketutils_run_all(void);

Purpose: Runs all registered socket utility examples

Return: Returns qscsdkc_status_success when all registered socket examples pass

qscsdkc_socketutils_start

qscsdkc_status qscsdkc_socketutils_start(void);

Purpose: Tests socket subsystem startup and shutdown

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_socketutils_address

qscsdkc_status qscsdkc_socketutils_address(void);

Purpose: Tests IPv4 and IPv6 address validation helpers

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_socketutils_create

qscsdkc_status qscsdkc_socketutils_create(void);

Purpose: Tests direct socketbase socket creation and cleanup

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_socketutils_options

qscsdkc_status qscsdkc_socketutils_options(void);

Purpose: Tests socket option configuration through socketbase

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_socketutils_blocking

qscsdkc_status qscsdkc_socketutils_blocking(void);

Purpose: Tests blocking and nonblocking socket mode changes

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_socketutils_tcp

qscsdkc_status qscsdkc_socketutils_tcp(void);

Purpose: Tests a loopback TCP client/server message exchange

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_socketutils_udp

qscsdkc_status qscsdkc_socketutils_udp(void);

Purpose: Tests UDP datagram socket creation and connectionless-state setup

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_socketutils_ready

qscsdkc_status qscsdkc_socketutils_ready(void);

Purpose: Tests send-ready and receive-ready polling helpers

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_socketutils_negative

qscsdkc_status qscsdkc_socketutils_negative(void);

Purpose: Tests negative socket input and failure handling examples

Return: Returns qscsdkc_status_success on success, or an error status on failure

26.38 QSCSDKC/sphincsplus_example.h (9 declarations)

qscsdkc_sphincsplus_run_all

qscsdkc_status qscsdkc_sphincsplus_run_all(void);

Purpose: Runs all SPHINCS+ example workflows registered in this module

Return: Returns qscsdkc_status_success when all registered SPHINCS+ examples pass; otherwise returns the first failing example status

qscsdkc_sphincsplus_keypair

qscsdkc_status qscsdkc_sphincsplus_keypair(void);

Purpose: Tests SPHINCS+ public/private key-pair generation

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_sphincsplus_sign

qscsdkc_status qscsdkc_sphincsplus_sign(void);

Purpose: Tests signing an in-memory message

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_sphincsplus_verify

qscsdkc_status qscsdkc_sphincsplus_verify(void);

Purpose: Tests verifying an in-memory signed message

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_sphincsplus_detached

qscsdkc_status qscsdkc_sphincsplus_detached(void);

Purpose: Tests detached signature creation and verification

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_sphincsplus_file

qscsdkc_status qscsdkc_sphincsplus_file(void);

Purpose: Tests signing and verifying file content

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_sphincsplus_publickey

qscsdkc_status qscsdkc_sphincsplus_publickey(void);

Purpose: Tests encoding and decoding an SPHINCS+ public key for transport

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_sphincsplus_domain

qscsdkc_status qscsdkc_sphincsplus_domain(void);

Purpose: Tests application-level context-bound SPHINCS+ signing

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_sphincsplus_negative

qscsdkc_status qscsdkc_sphincsplus_negative(void);

Purpose: Tests SPHINCS+ negative-path behavior for modified messages, signatures, and keys

Return: Returns qscsdkc_status_success on success, or an error status on failure

26.39 QSCSDKC/stringutils_example.h (11 declarations)

qscsdkc_stringutils_run_all

qscsdkc_status qscsdkc_stringutils_run_all(void);

Purpose: Runs all string utility example workflows registered in this module

Return: Returns qscsdkc_status_success when all registered string utility examples pass; otherwise returns the first failing example status

qscsdkc_stringutils_length

qscsdkc_status qscsdkc_stringutils_length(void);

Purpose: Tests qsc_stringutils_string_size and empty-string handling

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_stringutils_copy

qscsdkc_status qscsdkc_stringutils_copy(void);

Purpose: Tests qsc_stringutils_copy_string and qsc_stringutils_copy_substring

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_stringutils_concat

qscsdkc_status qscsdkc_stringutils_concat(void);

Purpose: Tests qsc_stringutils_concat_strings and qsc_stringutils_concat_and_copy

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_stringutils_compare

qscsdkc_status qscsdkc_stringutils_compare(void);

Purpose: Tests string comparison helpers

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_stringutils_find

qscsdkc_status qscsdkc_stringutils_find(void);

Purpose: Tests substring and character search helpers

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_stringutils_parse

qscsdkc_status qscsdkc_stringutils_parse(void);

Purpose: Tests split, insert, and substring extraction helpers

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_stringutils_format

qscsdkc_status qscsdkc_stringutils_format(void);

Purpose: Tests whitespace, formatting, and line-break helpers

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_stringutils_case

qscsdkc_status qscsdkc_stringutils_case(void);

Purpose: Tests uppercase, lowercase, and trim helpers

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_stringutils_numeric

qscsdkc_status qscsdkc_stringutils_numeric(void);

Purpose: Tests numeric string checks and integer conversions

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_stringutils_hex

qscsdkc_status qscsdkc_stringutils_hex(void);

Purpose: Tests single-byte hexadecimal string conversions

Return: Returns qscsdkc_status_success on success, or an error status on failure

26.40 QSCSDKC/timestamp_example.h (9 declarations)

qscsdkc_timestamp_run_all

qscsdkc_status qscsdkc_timestamp_run_all(void);

Purpose: Runs all timestamp utility example workflows registered in this module

Return: Returns qscsdkc_status_success when all registered timestamp utility examples pass; otherwise returns the first failing example status

qscsdkc_timestamp_current

qscsdkc_status qscsdkc_timestamp_current(void);

Purpose: Tests current date, current time, and current date-time formatting

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_timestamp_epoch

qscsdkc_status qscsdkc_timestamp_epoch(void);

Purpose: Tests seconds, milliseconds, and microseconds epoch-time retrieval

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_timestamp_convert

qscsdkc_status qscsdkc_timestamp_convert(void);

Purpose: Tests date-time string to seconds conversion and reverse conversion

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_timestamp_compare

qscsdkc_status qscsdkc_timestamp_compare(void);

Purpose: Tests date-time comparison and remaining-time calculation

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_timestamp_utc

qscsdkc_status qscsdkc_timestamp_utc(void);

Purpose: Tests UTC epoch-time retrieval

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_timestamp_stopwatch

qscsdkc_status qscsdkc_timestamp_stopwatch(void);

Purpose: Tests the high-level timerex stopwatch helper

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_timestamp_timerex

qscsdkc_status qscsdkc_timestamp_timerex(void);

Purpose: Tests timerex date, time, date-time, and elapsed-time helpers

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_timestamp_format

qscsdkc_status qscsdkc_timestamp_format(void);

Purpose: Tests manual struct tm formatting and parsing helpers

Return: Returns qscsdkc_status_success on success, or an error status on failure

26.41 QSCSDKC/tls_example.h (23 declarations)

qscsdkc_tls_run_all

qscsdkc_status qscsdkc_tls_run_all(void);

Purpose: Runs all TLS configuration and policy example tests. Executes the complete TLS example group, including certificate loading, client-context setup, server-context setup, loopback policy setup, echo configuration, hostname verification, mutual-TLS policy, ALPN, SNI, supported group configuration, fragmented-message handling, timeout policy, shutdown handling, and negative validation behavior

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating the first failed TLS example

qscsdkc_tls_certificate

qscsdkc_status qscsdkc_tls_certificate(void);

Purpose: Runs the TLS certificate binding example. Demonstrates how TLS client and server contexts consume X.509 certificate material. The example loads development certificate material, binds the server

certificate to the server-side TLS policy, and binds a root or trust anchor to the client-side verification policy

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating certificate loading or context-binding failure

qscsdkc_tls_client

qscsdkc_status qscsdkc_tls_client(void);

Purpose: Runs the TLS client-context setup example. Demonstrates how an application initializes a TLS client policy, including trust-anchor configuration, expected hostname assignment, supported group policy, signature-scheme policy, optional ALPN configuration, timeout settings, and context cleanup

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating client-context initialization or policy validation failure

qscsdkc_tls_server

qscsdkc_status qscsdkc_tls_server(void);

Purpose: Runs the TLS server-context setup example. Demonstrates how an application initializes a TLS server policy, including server-certificate binding, signing-key association, supported group policy, signature-scheme policy, optional ALPN/SNI configuration, and context cleanup

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating server-context initialization or policy validation failure

qscsdkc_tls_loopback

qscsdkc_status qscsdkc_tls_loopback(void);

Purpose: Runs the TLS loopback setup example. Demonstrates the policy and socket setup required for a local TLS loopback exchange. The example prepares client and server contexts suitable for a localhost connection and validates that the connection parameters are internally consistent before use

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating loopback setup or policy validation failure

qscsdkc_tls_echo

qscsdkc_status qscsdkc_tls_echo(void);

Purpose: Runs the TLS echo-application setup example. Demonstrates the application-level pattern used by a TLS echo service: a client sends protected application data, a server receives and processes the data, and the server returns a protected response. This configuration example documents the expected send/receive workflow without exposing TLS secrets

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating echo setup or application-data policy failure

qscsdkc_tls_hostname

qscsdkc_status qscsdkc_tls_hostname(void);

Purpose: Runs the TLS hostname-verification example. Demonstrates that a TLS client must verify the authenticated certificate identity against the expected DNS hostname. The example validates acceptance of the configured hostname and rejection of an incorrect hostname

Return: Returns qscsdkc_status_success if hostname policy behaves as expected; otherwise, returns an error status indicating hostname validation failure

qscsdkc_tls_mtlscert

qscsdkc_status qscsdkc_tls_mtlscert(void);

Purpose: Runs the TLS mutual-certificate authentication policy example. Demonstrates mutual TLS configuration, where the server requires a client certificate and the client also validates the server certificate. The example configures client and server authentication policy and validates rejection behavior when client authentication requirements are not met

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating mutual-TLS policy or certificate-authentication failure

qscsdkc_tls_alpn

qscsdkc_status qscsdkc_tls_alpn(void);

Purpose: Runs the TLS ALPN policy example. Demonstrates Application-Layer Protocol Negotiation configuration. The example configures client and server ALPN protocol lists, validates selection of a mutually supported application protocol, and demonstrates rejection or no-selection behavior when no acceptable protocol is available

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating ALPN configuration or negotiation-policy failure

qscsdkc_tls_sni

qscsdkc_status qscsdkc_tls_sni(void);

Purpose: Runs the TLS SNI certificate-selection example. Demonstrates Server Name Indication configuration. The example shows how a server can select certificate or identity policy based on the hostname requested by the client, while preserving the distinction between SNI-based server selection and client-side hostname verification

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating SNI configuration or selection-policy failure

qscsdkc_tls_groups

qscsdkc_status qscsdkc_tls_groups(void);

Purpose: Runs the TLS supported-groups and signature-policy example. Demonstrates how to configure TLS key-exchange groups and signature schemes for client and server contexts. The example validates that the configured policy contains compatible groups and signature schemes and documents how applications should avoid brittle assumptions about fixed group indexes

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating group-policy or signature-policy failure

qscsdkc_tls_fragments

qscsdkc_status qscsdkc_tls_fragments(void);

Purpose: Runs the TLS fragmented-application-data example. Demonstrates application-data handling for messages that may span multiple TLS records or socket reads. The example documents the distinction between TLS-protected byte-stream transport and application-level message framing

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating fragmented-message setup or validation failure

qscsdkc_tls_timeout

qscsdkc_status qscsdkc_tls_timeout(void);

Purpose: Runs the TLS timeout-policy example. Demonstrates timeout configuration for TLS connections, including connection, handshake, read, and write timeout behavior where supported by the QSC TLS socket wrapper and socket utilities

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating timeout-policy configuration or validation failure

qscsdkc_tls_shutdown

qscsdkc_status qscsdkc_tls_shutdown(void);

Purpose: Runs the TLS shutdown-policy example. Demonstrates orderly TLS connection shutdown and resource cleanup. The example documents the expected sequence for closing TLS protocol state, closing the underlying socket, and disposing of sensitive connection state

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating shutdown-policy or cleanup failure

qscsdkc_tls_negative

qscsdkc_status qscsdkc_tls_negative(void);

Purpose: Runs the TLS negative-policy example. Demonstrates secure rejection behavior for invalid TLS configurations and policy mismatches. The example covers cases such as hostname mismatch, untrusted identity policy, missing client authentication where required, unsupported ALPN, incompatible groups, or invalid connection state where supported by the example harness

Return: Returns qscsdkc_status_success if invalid cases are rejected as expected; otherwise, returns an error status indicating unexpected acceptance or negative-test failure

qscsdkc_tls_live_run_all

qscsdkc_status qscsdkc_tls_live_run_all(void);

Purpose: Runs all live TLS loopback example tests. Executes the complete live TLS loopback group, including localhost client/server setup, live echo exchange, ALPN and SNI live-policy variants, mutual-TLS behavior, live shutdown cleanup, and live negative-path validation

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating the first failed live TLS example

qscsdkc_tls_live_loopback

qscsdkc_status qscsdkc_tls_live_loopback(void);

Purpose: Runs the live TLS loopback connection example. Demonstrates a localhost TLS connection using a QSC TLS client and server in the same example harness. The example starts a local server, connects a client, performs the configured TLS exchange, and validates that the loopback path completes successfully

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating live loopback setup, handshake, or cleanup failure

qscsdkc_tls_live_echo

qscsdkc_status qscsdkc_tls_live_echo(void);

Purpose: Runs the live TLS echo exchange example. Demonstrates a complete protected application-data exchange over a live localhost TLS connection. The client sends an application message to the

server, the server reads the protected message and sends a response, and the client verifies the echoed or returned data

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating live echo connection, send, receive, comparison, or cleanup failure

qscsdkc_tls_live_alpn

qscsdkc_status qscsdkc_tls_live_alpn(void);

Purpose: Runs the live TLS ALPN negotiation example. Demonstrates ALPN negotiation during a live localhost TLS connection. The example configures compatible client and server protocol lists and verifies that the selected application protocol matches the expected policy

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating live ALPN negotiation or validation failure

qscsdkc_tls_live_sni

qscsdkc_status qscsdkc_tls_live_sni(void);

Purpose: Runs the live TLS SNI selection example. Demonstrates SNI handling during a live localhost TLS connection. The client supplies a server name indication value and the server applies the matching certificate or identity-selection policy for that hostname

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating live SNI selection or validation failure

qscsdkc_tls_live_mtls

qscsdkc_status qscsdkc_tls_live_mtls(void);

Purpose: Runs the live mutual-TLS example. Demonstrates mutual-certificate authentication over a live localhost TLS connection. The example configures server-side client-certificate requirements, client-side certificate presentation, and validates acceptance or rejection according to the configured mTLS policy

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating live mutual-TLS authentication or policy failure

qscsdkc_tls_live_shutdown

qscsdkc_status qscsdkc_tls_live_shutdown(void);

Purpose: Runs the live TLS shutdown example. Demonstrates clean shutdown of a live TLS loopback connection. The example closes TLS protocol state, releases socket resources, joins any worker thread used by the loopback harness, and clears sensitive temporary state

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating live shutdown or cleanup failure

qscsdkc_tls_live_negative

qscsdkc_status qscsdkc_tls_live_negative(void);

Purpose: Runs the live TLS negative-path example. Demonstrates rejection of invalid live TLS connection attempts. The example verifies that a live connection fails safely when policy checks fail, such as wrong hostname, missing client certificate when mTLS is required, unsupported ALPN, or other configured negative-path conditions

Return: Returns qscsdkc_status_success if invalid live cases are rejected as expected; otherwise, returns an error status indicating unexpected acceptance or live negative-test failure

26.42 QSCSDKC/x509_author_example.h (9 declarations)

qscsdkc_x509_author_run_all

qscsdkc_status qscsdkc_x509_author_run_all(void);

Purpose: Runs all registered X.509 authoring examples

Return: Returns qscsdkc_status_success when all examples pass; otherwise returns the first failing status

qscsdkc_x509_author_builder

qscsdkc_status qscsdkc_x509_author_builder(void);

Purpose: Tests constructing a certificate builder with issuer, subject, SPKI, and validity fields

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_x509_author_root

qscsdkc_status qscsdkc_x509_author_root(void);

Purpose: Tests creating and decoding a self-signed ML-DSA root certificate

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_x509_author_csr

qscsdkc_status qscsdkc_x509_author_csr(void);

Purpose: Tests creating, signing, and decoding a PKCS#10 CSR

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_x509_author_issue

qscsdkc_status qscsdkc_x509_author_issue(void);

Purpose: Tests issuing a TLS server certificate from a CSR using a generated root issuer

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_x509_author_pem

qscsdkc_status qscsdkc_x509_author_pem(void);

Purpose: Tests DER-to-PEM conversion for a generated certificate

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_x509_author_key

qscsdkc_status qscsdkc_x509_author_key(void);

Purpose: Tests PKCS#8 PEM export of a generated ML-DSA private key

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_x509_author_profile

qscsdkc_status qscsdkc_x509_author_profile(void);

Purpose: Tests built-in X.509 issuance profile application

Return: Returns qscsdkc_status_success on success, or an error status on failure

qscsdkc_x509_author_negative

qscsdkc_status qscsdkc_x509_author_negative(void);

Purpose: Tests negative authoring behavior for incomplete certificate builders

Return: Returns qscsdkc_status_success on success, or an error status on failure

26.43 QSCSDKC/x509_example.h (13 declarations)

qscsdkc_x509_run_all

qscsdkc_status qscsdkc_x509_run_all(void);

Purpose: Runs all X.509 certificate validation example tests. Executes the complete X.509 example group, including DER parsing, certificate inspection, root trust-anchor handling, chain verification, hostname validation, validity-period checks, key-usage checks, fingerprint generation, public-key extraction, issuer-signature verification, complete TLS-server certificate validation, and negative validation tests

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating the first failed X.509 example

qscsdkc_x509_parse

qscsdkc_status qscsdkc_x509_parse(void);

Purpose: Runs the X.509 DER certificate parsing example. Demonstrates how to parse a DER-encoded X.509 certificate into the QSC certificate representation. The example validates that the input certificate can be decoded and that malformed or incomplete certificate data is rejected

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating certificate parsing failure

qscsdkc_x509_inspect

qscsdkc_status qscsdkc_x509_inspect(void);

Purpose: Runs the X.509 certificate inspection example. Demonstrates how an application can inspect fields from a parsed certificate, such as issuer, subject, serial number, validity period, public-key information, signature algorithm, and certificate extensions exposed by the QSC X.509 API

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating certificate parsing or inspection failure

qscsdkc_x509_root

qscsdkc_status qscsdkc_x509_root(void);

Purpose: Runs the X.509 root trust-anchor example. Demonstrates how to load and treat a root certificate as a trust anchor for application validation. The example emphasizes that parsing a certificate does not make it trusted; trust is established by application policy or by a configured trust store

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating root certificate loading or trust-anchor validation failure

qscsdkc_x509_chain

qscsdkc_status qscsdkc_x509_chain(void);

Purpose: Runs the X.509 certificate-chain verification example. Demonstrates how to verify a certificate chain using a leaf certificate, an intermediate certificate, and a trusted root certificate. The example validates issuer relationships, certificate signatures, CA constraints, and chain trust according to the QSC X.509 validation path used by the SDKC example

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating chain verification failure

qscsdkc_x509_hostname

qscsdkc_status qscsdkc_x509_hostname(void);

Purpose: Runs the X.509 hostname validation example. Demonstrates how a TLS-style application verifies that a certificate identifies the expected DNS hostname. The example validates acceptance of a

matching hostname and rejection of an incorrect hostname, using the hostname checking behavior exposed by the QSC X.509 API

Return: Returns qscsdkc_status_success if hostname validation behaves as expected; otherwise, returns an error status indicating hostname validation failure

qscsdkc_x509_validity

qscsdkc_status qscsdkc_x509_validity(void);

Purpose: Runs the X.509 validity-period example. Demonstrates how an application checks a certificate's not-before and not-after validity fields against the current time. The example shows that expired certificates and not-yet-valid certificates must be rejected even when their signatures are otherwise valid

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating validity-period checking failure

qscsdkc_x509_usage

qscsdkc_status qscsdkc_x509_usage(void);

Purpose: Runs the X.509 key-usage validation example. Demonstrates how an application checks whether a certificate is suitable for a specific purpose, such as TLS server authentication, TLS client authentication, digital signature use, or certificate-authority signing. The example emphasizes that a valid chain does not automatically make a certificate valid for every application use

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating key-usage or extended-key-usage validation failure

qscsdkc_x509_fingerprint

qscsdkc_status qscsdkc_x509_fingerprint(void);

Purpose: Runs the X.509 certificate fingerprint example. Demonstrates how to compute a certificate fingerprint, typically using a SHA2-256 digest of the DER certificate bytes. The example shows how a fingerprint can be used as a stable certificate identifier for logging, manual verification, or pinning workflows

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating fingerprint generation or comparison failure

qscsdkc_x509_publickey

qscsdkc_status qscsdkc_x509_publickey(void);

Purpose: Runs the X.509 public-key extraction example. Demonstrates how to extract public-key material from a parsed certificate. The example emphasizes that the extracted key should not be trusted until the certificate chain, validity period, hostname, and usage policy have been successfully validated

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating public-key extraction failure

qscsdkc_x509_sign

qscsdkc_status qscsdkc_x509_sign(void);

Purpose: Runs the X.509 certificate-signature verification example. Demonstrates how to verify that a subject certificate was signed by its issuer certificate. This example covers issuer-signature validation only and distinguishes signature verification from complete certificate trust validation

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating issuer-signature verification failure

qscsdkc_x509_verify

qscsdkc_status qscsdkc_x509_verify(void);

Purpose: Runs the complete X.509 TLS-server certificate verification example. Demonstrates a full application-style certificate validation path for a TLS server certificate. The example combines chain validation, root trust-anchor policy, validity-period checks, certificate usage checks, public-key suitability, and hostname verification into a single accept-or-reject decision

Return: Returns qscsdkc_status_success on success; otherwise, returns an error status indicating complete certificate verification failure

qscsdkc_x509_negative

qscsdkc_status qscsdkc_x509_negative(void);

Purpose: Runs the X.509 negative-validation example. Demonstrates secure rejection behavior for invalid certificate inputs and validation failures. The example covers cases such as malformed certificate data, wrong issuer relationships, invalid signatures, expired or not-yet- valid certificates, wrong hostname, unsuitable usage, and untrusted roots where supported by the example harness

Return: Returns qscsdkc_status_success if invalid cases are rejected as expected; otherwise, returns an error status indicating unexpected acceptance or negative-test failure

27: QSCSDKTest Public C Function Reference

This chapter reproduces 131 function prototypes from 30 declared project headers. For QSCSDKC these are teaching/diagnostic example entry points; for QSCSDKTest they are qualification-workbench functions. The native QSC library is external, and none of these declarations should be interpreted as the complete QSC crypto API. Parameter and return descriptions are based on adjacent header Doxygen when supplied; otherwise a purpose summary is explicitly inferred from the function name and implementation category.

27.1 QSCSDKTest/qscsdk_benchmark.h (7 declarations)

qscsdk_benchmark_manager_initialize

bool qscsdk_benchmark_manager_initialize(qscsdk_benchmark_manager* manager);

Source-derived role: Perform initialize on the associated SDK state, preserving the caller's ownership and native lifetime contract.

qscsdk_benchmark_manager_dispose

void qscsdk_benchmark_manager_dispose(qscsdk_benchmark_manager* manager);

Source-derived role: Perform dispose on the associated SDK state, preserving the caller's ownership and native lifetime contract.

qscsdk_benchmark_manager_start

bool qscsdk_benchmark_manager_start(qscsdk_benchmark_manager* manager, const char* benchmark_id, size_t iterations, char* detail, size_t detail_len);

Source-derived role: Perform start on the associated SDK state, preserving the caller's ownership and native lifetime contract.

qscsdk_benchmark_manager_cancel

bool qscsdk_benchmark_manager_cancel(qscsdk_benchmark_manager* manager, char* detail, size_t detail_len);

Source-derived role: Perform cancel on the associated SDK state, preserving the caller's ownership and native lifetime contract.

qscsdk_benchmark_manager_status

bool qscsdk_benchmark_manager_status(qscsdk_benchmark_manager* manager, qscsdk_benchmark_snapshot* snapshot);

Source-derived role: SDK qualification platform helper for benchmark manager status. Validate status, pointers and output length at each call.

qscsdk_benchmark_definitions

const qscsdk_benchmark_definition* qscsdk_benchmark_definitions(size_t* count);

Source-derived role: SDK qualification platform helper for benchmark definitions. Validate status, pointers and output length at each call.

qscsdk_benchmark_state_name

const char* qscsdk_benchmark_state_name(qscsdk_benchmark_state state);

Source-derived role: SDK qualification platform helper for benchmark state name. Validate status, pointers and output length at each call.

27.2 QSCSDKTest/qscsdk_catalog.h (1 declarations)

qscsdk_catalog_evidence_name

const char* qscsdk_catalog_evidence_name(uint32_t flag);

Purpose: Convert a single evidence bit to its stable display name

Return: [const char*] Stable evidence-class name

27.3 QSCSDKTest/qscsdk_cli.h (1 declarations)

qscsdk_cli_parse

bool qscsdk_cli_parse(int32_t argc, char** argv, qscsdk_cli_options* options);

Source-derived role: SDK qualification platform helper for cli parse. Validate status, pointers and output length at each call.

27.4 QSCSDKTest/qscsdk_console.h (5 declarations)

qscsdk_console_print_title

void qscsdk_console_print_title(void);

Purpose: Print the SDK testing-platform title

qscsdk_console_print_help

void qscsdk_console_print_help(void);

Purpose: Print command-line usage

qscsdk_console_print_definition

void qscsdk_console_print_definition(const qscsdk_test_definition* definition);

Purpose: Print one registered test definition

qscsdk_console_print_result

void qscsdk_console_print_result(const qscsdk_test_result* result);

Purpose: Print one completed structured test result

qscsdk_console_print_summary

void qscsdk_console_print_summary(const qscsdk_run_summary* summary);

Purpose: Print aggregate run counts and timing

27.5 QSCSDKTest/qscsdk_coverage.h (1 declarations)

qscsdk_coverage_collect

bool qscsdk_coverage_collect(qscsdk_coverage_snapshot* snapshot);

Source-derived role: SDK qualification platform helper for coverage collect. Validate status, pointers and output length at each call.

27.6 QSCSDKTest/qscsdk_crypto_tests.h (6 declarations)

qscsdk_crypto_test_mlkem

qscsdk_test_status qscsdk_crypto_test_mlkem(const qscsdk_test_context* context, char* detail, size_t detail_len);

Purpose: Qualify ML-KEM key generation, encapsulation, decapsulation, and implicit rejection

qscsdk_crypto_test_mldsa

qscsdk_test_status qscsdk_crypto_test_mldsa(const qscsdk_test_context* context, char* detail, size_t detail_len);

Purpose: Qualify ML-DSA signing, recovery, verification, and tamper rejection

qscsdk_crypto_test_slhdsa

qscsdk_test_status qscsdk_crypto_test_slhdsa(const qscsdk_test_context* context, char* detail, size_t detail_len);

Purpose: Qualify SLH-DSA signing, recovery, verification, and tamper rejection

qscsdk_crypto_test_ecdh

qscsdk_test_status qscsdk_crypto_test_ecdh(const qscsdk_test_context* context, char* detail, size_t detail_len);

Purpose: Qualify the active ECDH parameter set using reciprocal shared-secret derivation

qscsdk_crypto_test_ecdsa

qscsdk_test_status qscsdk_crypto_test_ecdsa(const qscsdk_test_context* context, char* detail, size_t detail_len);

Purpose: Qualify ECDSA signing, message recovery, verification, and tamper rejection

qscsdk_crypto_test_eddsa

qscsdk_test_status qscsdk_crypto_test_eddsa(const qscsdk_test_context* context, char* detail, size_t detail_len);

Purpose: Qualify EdDSA signing, message recovery, verification, and tamper rejection

27.7 QSCSDKTest/qscsdk_environment.h (3 declarations)

qscsdk_environment_initialize

bool qscsdk_environment_initialize(qscsdk_run_environment* environment, uint64_t seed);

Purpose: Initialize immutable environment metadata for a new qualification run

Return: [bool] True when the environment was initialized

qscsdk_environment_complete

void qscsdk_environment_complete(qscsdk_run_environment* environment);

Purpose: Mark the environment record with the current UTC completion time

qscsdk_environment_current_utc

bool qscsdk_environment_current_utc(char* output, size_t output_len);

Purpose: Write the current UTC time in ISO-8601 format

Return: [bool] True when a timestamp was written

27.8 QSCSDKTest/qscsdk_evidence.h (8 declarations)

qscsdk_evidence_root_from_executable

bool qscsdk_evidence_root_from_executable(const char* executable_path, char* output, size_t output_len);

Source-derived role: Produce, load or export SDK qualification evidence according to the specified file paths, capacities and integrity checks.

qscsdk_evidence_source_root_from_executable

bool qscsdk_evidence_source_root_from_executable(const char* executable_path, char* output, size_t output_len);

Source-derived role: Produce, load or export SDK qualification evidence according to the specified file paths, capacities and integrity checks.

qscsdk_evidence_generate_report

bool qscsdk_evidence_generate_report(const char* root, qscsdk_run_manager* manager, qscsdk_benchmark_manager* benchmark_manager, qscsdk_fuzz_manager* fuzz_manager, const char* run_id, qscsdk_evidence_bundle* bundle, char* detail, size_t detail_len);

Source-derived role: Produce, load or export SDK qualification evidence according to the specified file paths, capacities and integrity checks.

qscsdk_evidence_export_run

bool qscsdk_evidence_export_run(const char* root, const char* source_root, qscsdk_run_manager* manager, qscsdk_benchmark_manager* benchmark_manager, qscsdk_fuzz_manager* fuzz_manager, const char* run_id, qscsdk_evidence_bundle* bundle, char* detail, size_t detail_len);

Source-derived role: Produce, load or export SDK qualification evidence according to the specified file paths, capacities and integrity checks.

qscsdk_evidence_list

size_t qscsdk_evidence_list(const char* root, qscsdk_evidence_bundle* records, size_t capacity);

Source-derived role: Produce, load or export SDK qualification evidence according to the specified file paths, capacities and integrity checks.

qscsdk_evidence_resolve_file

bool qscsdk_evidence_resolve_file(const char* root, const char* run_id, const char* file_name, char* output, size_t output_len);

Source-derived role: Produce, load or export SDK qualification evidence according to the specified file paths, capacities and integrity checks.

qscsdk_evidence_read_file

bool qscsdk_evidence_read_file(const char* path, char** output, size_t* output_len);

Source-derived role: Produce, load or export SDK qualification evidence according to the specified file paths, capacities and integrity checks.

qscsdk_evidence_free_file

void qscsdk_evidence_free_file(char* output);

Source-derived role: Produce, load or export SDK qualification evidence according to the specified file paths, capacities and integrity checks.

27.9 QSCSDKTest/qscsdk_executive_report.h (1 declarations)

qscsdk_executive_report_write

bool qscsdk_executive_report_write(const char* path, const qscsdk_test_result* results, size_t result_count, const qscsdk_run_summary* summary, const qscsdk_run_environment* environment, qscsdk_run_state state, size_t selected_count, size_t completed_count, const qscsdk_benchmark_snapshot* benchmark, const qscsdk_fuzz_snapshot* fuzz, const qscsdk_coverage_snapshot* coverage);

Purpose: Write a self-contained executive qualification report

Return: [bool] True when the report was written completely

27.10 QSCSDKTest/qscsdk_fuzz.h (8 declarations)

qscsdk_fuzz_manager_initialize

bool qscsdk_fuzz_manager_initialize(qscsdk_fuzz_manager* manager, const char* executable_path);

Source-derived role: Perform initialize on the associated SDK state, preserving the caller's ownership and native lifetime contract.

qscsdk_fuzz_manager_dispose

void qscsdk_fuzz_manager_dispose(qscsdk_fuzz_manager* manager);

Source-derived role: Perform dispose on the associated SDK state, preserving the caller's ownership and native lifetime contract.

qscsdk_fuzz_manager_start

bool qscsdk_fuzz_manager_start(qscsdk_fuzz_manager* manager, const char* target_id, size_t iterations, uint64_t seed, char* detail, size_t detail_len);

Source-derived role: Perform start on the associated SDK state, preserving the caller's ownership and native lifetime contract.

qscsdk_fuzz_manager_cancel

bool qscsdk_fuzz_manager_cancel(qscsdk_fuzz_manager* manager, char* detail, size_t detail_len);

Source-derived role: Perform cancel on the associated SDK state, preserving the caller's ownership and native lifetime contract.

qscsdk_fuzz_manager_status

bool qscsdk_fuzz_manager_status(qscsdk_fuzz_manager* manager, qscsdk_fuzz_snapshot* snapshot);

Source-derived role: SDK qualification platform helper for fuzz manager status. Validate status, pointers and output length at each call.

qscsdk_fuzz_worker_execute

int32_t qscsdk_fuzz_worker_execute(const char* target_id, size_t iterations, uint64_t seed, const char* status_path);

Source-derived role: SDK qualification platform helper for fuzz worker execute. Validate status, pointers and output length at each call.

qscsdk_fuzz_targets

const qscsdk_fuzz_target* qscsdk_fuzz_targets(size_t* count);

Source-derived role: SDK qualification platform helper for fuzz targets. Validate status, pointers and output length at each call.

qscsdk_fuzz_state_name

const char* qscsdk_fuzz_state_name(qscsdk_fuzz_state state);

Source-derived role: SDK qualification platform helper for fuzz state name. Validate status, pointers and output length at each call.

27.11 QSCSDKTest/qscsdk_integrity.h (1 declarations)

qscsdk_integrity_file_sha256

bool qscsdk_integrity_file_sha256(const char* path, char* output, size_t output_len, size_t* file_size);

Source-derived role: SDK qualification platform helper for integrity file sha256. Validate status, pointers and output length at each call.

27.12 QSCSDKTest/qscsdk_path.h (3 declarations)

qscsdk_path_executable_file

bool qscsdk_path_executable_file(const char* executable_hint, char* output, size_t output_len);

Purpose: Narrow platform path helpers used by the standalone SDK harness

qscsdk_path_executable_directory

bool qscsdk_path_executable_directory(const char* executable_hint, char* output, size_t output_len);

Source-derived role: SDK qualification platform helper for path executable directory. Validate status, pointers and output length at each call.

qscsdk_path_join

bool qscsdk_path_join(char* output, size_t output_len, const char* left, const char* right);

Source-derived role: SDK qualification platform helper for path join. Validate status, pointers and output length at each call.

27.13 QSCSDKTest/qscsdk_registry.h (2 declarations)

qscsdk_registry_get

const qscsdk_test_definition* qscsdk_registry_get(size_t* count);

Purpose: Return the immutable registry and its entry count

Return: [const struct*] Pointer to the first registry entry

qscsdk_registry_find

const qscsdk_test_definition* qscsdk_registry_find(const char* id);

Purpose: Find a registered test by stable identifier

Return: [const struct*] Matching definition, or NULL when not found

27.14 QSCSDKTest/qscsdk_report.h (2 declarations)

qscsdk_report_write_json

bool qscsdk_report_write_json(const char* path, const qscsdk_test_result* results, size_t count, const qscsdk_run_summary* summary, const qscsdk_run_environment* environment);

Purpose: Persistent report and catalog writers for SDK qualification evidence

qscsdk_report_write_catalog_json

bool qscsdk_report_write_catalog_json(const char* path, const qscsdk_test_definition* definitions, size_t count);

Source-derived role: Produce, load or export SDK qualification evidence according to the specified file paths, capacities and integrity checks.

27.15 QSCSDKTest/qscsdk_report_theme.h (1 declarations)

qscsdk_report_theme_write

bool qscsdk_report_theme_write(FILE* stream);

Purpose: Write the CBOM-derived light executive-report stylesheet. The stylesheet is local-file safe and contains no remote fonts, images, or network dependencies. It includes responsive, accessibility, and print rules

Parameters: stream: [FILE*] The output HTML stream.

Return: [bool] Returns true when the complete stylesheet is written

27.16 QSCSDKTest/qscsdk_result.h (11 declarations)

qscsdk_result_initialize

void qscsdk_result_initialize(qscsdk_test_result* result, const qscsdk_test_definition* definition, const qscsdk_test_context* context);

Purpose: Structured test-result and execution-evidence helpers

qscsdk_result_set_detail

void qscsdk_result_set_detail(qscsdk_test_result* result, const char* detail);

Source-derived role: SDK qualification platform helper for result set detail. Validate status, pointers and output length at each call.

qscsdk_result_set_input_summary

void qscsdk_result_set_input_summary(qscsdk_test_result* result, const char* summary);

Source-derived role: SDK qualification platform helper for result set input summary. Validate status, pointers and output length at each call.

qscsdk_result_set_expected_summary

void qscsdk_result_set_expected_summary(qscsdk_test_result* result, const char* summary);

Source-derived role: SDK qualification platform helper for result set expected summary. Validate status, pointers and output length at each call.

qscsdk_result_set_observed_summary

void qscsdk_result_set_observed_summary(qscsdk_test_result* result, const char* summary);

Source-derived role: SDK qualification platform helper for result set observed summary. Validate status, pointers and output length at each call.

qscsdk_result_set_case_counts

void qscsdk_result_set_case_counts(qscsdk_test_result* result, size_t total, size_t passed, size_t failed);

Source-derived role: SDK qualification platform helper for result set case counts. Validate status, pointers and output length at each call.

qscsdk_result_record_assertion

bool qscsdk_result_record_assertion(qscsdk_test_result* result, const char* id, const char* text, qscsdk_assertion_status status);

Source-derived role: SDK qualification platform helper for result record assertion. Validate status, pointers and output length at each call.

qscsdk_result_add_artifact

bool qscsdk_result_add_artifact(qscsdk_test_result* result, const char* type, const char* label, const char* path, const char* sha256);

Source-derived role: SDK qualification platform helper for result add artifact. Validate status, pointers and output length at each call.

qscsdk_result_status_name

const char* qscsdk_result_status_name(qscsdk_test_status status);

Source-derived role: SDK qualification platform helper for result status name. Validate status, pointers and output length at each call.

qscsdk_result_type_name

const char* qscsdk_result_type_name(qscsdk_test_type type);

Source-derived role: SDK qualification platform helper for result type name. Validate status, pointers and output length at each call.

qscsdk_result_assertion_status_name

const char* qscsdk_result_assertion_status_name(qscsdk_assertion_status status);

Source-derived role: SDK qualification platform helper for result assertion status name. Validate status, pointers and output length at each call.

27.17 QSCSDKTest/qscsdk_run_manager.h (11 declarations)

qscsdk_run_manager_initialize

bool qscsdk_run_manager_initialize(qscsdk_run_manager* manager);

Source-derived role: Perform initialize on the associated SDK state, preserving the caller’s ownership and native lifetime contract.

qscsdk_run_manager_dispose

void qscsdk_run_manager_dispose(qscsdk_run_manager* manager);

Source-derived role: Perform dispose on the associated SDK state, preserving the caller's ownership and native lifetime contract.

qscsdk_run_manager_start

bool qscsdk_run_manager_start(qscsdk_run_manager* manager, bool select_all, const char* const* test_ids, size_t test_count, uint64_t seed, char* detail, size_t detail_len);

Source-derived role: Perform start on the associated SDK state, preserving the caller's ownership and native lifetime contract.

qscsdk_run_manager_cancel

bool qscsdk_run_manager_cancel(qscsdk_run_manager* manager, char* detail, size_t detail_len);

Source-derived role: Perform cancel on the associated SDK state, preserving the caller's ownership and native lifetime contract.

qscsdk_run_manager_status

bool qscsdk_run_manager_status(qscsdk_run_manager* manager, qscsdk_run_status_snapshot* snapshot);

Source-derived role: SDK qualification platform helper for run manager status. Validate status, pointers and output length at each call.

qscsdk_run_manager_snapshot

bool qscsdk_run_manager_snapshot(qscsdk_run_manager* manager, const char* run_id, qscsdk_run_status_snapshot* snapshot);

Source-derived role: SDK qualification platform helper for run manager snapshot. Validate status, pointers and output length at each call.

qscsdk_run_manager_result_briefs

size_t qscsdk_run_manager_result_briefs(qscsdk_run_manager* manager, const char* run_id, qscsdk_run_result_brief* records, size_t capacity);

Source-derived role: SDK qualification platform helper for run manager result briefs. Validate status, pointers and output length at each call.

qscsdk_run_manager_history

size_t qscsdk_run_manager_history(qscsdk_run_manager* manager, qscsdk_run_history_record* records, size_t capacity);

Source-derived role: SDK qualification platform helper for run manager history. Validate status, pointers and output length at each call.

qscsdk_run_manager_result

bool qscsdk_run_manager_result(qscsdk_run_manager* manager, const char* run_id, const char* test_id, qscsdk_test_result* result, qscsdk_run_environment* environment, qscsdk_run_state* state);

Source-derived role: SDK qualification platform helper for run manager result. Validate status, pointers and output length at each call.

qscsdk_run_manager_copy_results

size_t qscsdk_run_manager_copy_results(qscsdk_run_manager* manager, const char* run_id, qscsdk_test_result* results, size_t capacity, qscsdk_run_environment* environment, qscsdk_run_summary* summary, qscsdk_run_state* state);

Source-derived role: SDK qualification platform helper for run manager copy results. Validate status, pointers and output length at each call.

qscsdk_run_manager_state_name

const char* qscsdk_run_manager_state_name(qscsdk_run_state state);

Source-derived role: SDK qualification platform helper for run manager state name. Validate status, pointers and output length at each call.

27.18 QSCSDKTest/qscsdk_runner.h (2 declarations)

qscsdk_runner_execute

bool qscsdk_runner_execute(const qscsdk_test_definition* definition, const qscsdk_test_context* context, qscsdk_test_result* result);

Purpose: Execute a single registered test

Return: [bool] True when the runner completed the invocation

qscsdk_runner_summarize

void qscsdk_runner_summarize(const qscsdk_test_result* results, size_t count, qscsdk_run_summary* summary);

Purpose: Summarize an array of completed test results

27.19 QSCSDKTest/qscsdk_tests.h (6 declarations)

qscsdk_test_selftest_aes

qscsdk_test_status qscsdk_test_selftest_aes(const qscsdk_test_context* context, char* detail, size_t detail_len);

Purpose: Execute the QSC AES internal self-test

qscsdk_test_selftest_chacha

qscsdk_test_status qscsdk_test_selftest_chacha(const qscsdk_test_context* context, char* detail, size_t detail_len);

Purpose: Execute the QSC ChaCha internal self-test

qscsdk_test_selftest_csx

qscsdk_test_status qscsdk_test_selftest_csx(const qscsdk_test_context* context, char* detail, size_t detail_len);

Purpose: Execute the QSC CSX internal self-test

qscsdk_test_selftest_rcs

qscsdk_test_status qscsdk_test_selftest_rcs(const qscsdk_test_context* context, char* detail, size_t detail_len);

Purpose: Execute the QSC RCS internal self-test

qscsdk_test_selftest_sha2

qscsdk_test_status qscsdk_test_selftest_sha2(const qscsdk_test_context* context, char* detail, size_t detail_len);

Purpose: Execute the QSC SHA-2 internal self-test

qscsdk_test_selftest_sha3

qscsdk_test_status qscsdk_test_selftest_sha3(const qscsdk_test_context* context, char* detail, size_t detail_len);

Purpose: Execute the QSC SHA-3 internal self-test

27.20 QSCSDKTest/qscsdk_tls_tests.h (6 declarations)

qscsdk_tls_test_codec

qscsdk_test_status qscsdk_tls_test_codec(const qscsdk_test_context* context, char* detail, size_t detail_len);

Purpose: Qualify primitive TLS codec round trips and bounds enforcement

qscsdk_tls_test_extensions

qscsdk_test_status qscsdk_tls_test_extensions(const qscsdk_test_context* context, char* detail, size_t detail_len);

Purpose: Qualify supported_versions and SNI extension encoding/decoding

qscsdk_tls_test_transcript

qscsdk_test_status qscsdk_tls_test_transcript(const qscsdk_test_context* context, char* detail, size_t detail_len);

Purpose: Qualify transcript hashing against the direct SHA-256 implementation

qscsdk_tls_test_record

qscsdk_test_status qscsdk_tls_test_record(const qscsdk_test_context* context, char* detail, size_t detail_len);

Purpose: Qualify plaintext record encoding, decoding, and span detection

qscsdk_tls_test_finished

qscsdk_test_status qscsdk_tls_test_finished(const qscsdk_test_context* context, char* detail, size_t detail_len);

Purpose: Qualify Finished computation, verification, and tamper rejection

qscsdk_tls_test_pq_capabilities

qscsdk_test_status qscsdk_tls_test_pq_capabilities(const qscsdk_test_context* context, char* detail, size_t detail_len);

Purpose: Qualify active post-quantum TLS group and signature-scheme metadata

27.21 QSCSDKTest/qscsdk_vector_tests.h (8 declarations)

qscsdk_vector_test_integrity

qscsdk_test_status qscsdk_vector_test_integrity(const qscsdk_test_context* context, char* detail, size_t detail_len);

Purpose: Vector-driven qualification tests for the standalone SDK harness

qscsdk_vector_test_aes_cbc

qscsdk_test_status qscsdk_vector_test_aes_cbc(const qscsdk_test_context* context, char* detail, size_t detail_len);

Source-derived role: SDK qualification platform helper for vector test aes cbc. Validate status, pointers and output length at each call.

qscsdk_vector_test_sha256_short

qscsdk_test_status qscsdk_vector_test_sha256_short(const qscsdk_test_context* context, char* detail, size_t detail_len);

Source-derived role: SDK qualification platform helper for vector test sha256 short. Validate status, pointers and output length at each call.

qscsdk_vector_test_sha3_256_short

qscsdk_test_status qscsdk_vector_test_sha3_256_short(const qscsdk_test_context* context, char* detail, size_t detail_len);

Source-derived role: SDK qualification platform helper for vector test sha3 256 short. Validate status, pointers and output length at each call.

qscsdk_vector_test_mlkem_keygen

qscsdk_test_status qscsdk_vector_test_mlkem_keygen(const qscsdk_test_context* context, char* detail, size_t detail_len);

Source-derived role: SDK qualification platform helper for vector test mlkem keygen. Validate status, pointers and output length at each call.

qscsdk_vector_test_mlkem_encap

qscsdk_test_status qscsdk_vector_test_mlkem_encap(const qscsdk_test_context* context, char* detail, size_t detail_len);

Source-derived role: SDK qualification platform helper for vector test mlkem encap. Validate status, pointers and output length at each call.

qscsdk_vector_test_mldsa_keygen

qscsdk_test_status qscsdk_vector_test_mldsa_keygen(const qscsdk_test_context* context, char* detail, size_t detail_len);

Source-derived role: SDK qualification platform helper for vector test mldsa keygen. Validate status, pointers and output length at each call.

qscsdk_vector_test_slhdsa_keygen

qscsdk_test_status qscsdk_vector_test_slhdsa_keygen(const qscsdk_test_context* context, char* detail, size_t detail_len);

Source-derived role: SDK qualification platform helper for vector test slhdsa keygen. Validate status, pointers and output length at each call.

27.22 QSCSDKTest/qscsdk_vectors.h (2 declarations)

qscsdk_vectors_resolve_path

bool qscsdk_vectors_resolve_path(const qscsdk_vector_source* vector, char* path, size_t path_len);

Source-derived role: SDK qualification platform helper for vectors resolve path. Validate status, pointers and output length at each call.

qscsdk_vectors_find

const qscsdk_vector_source* qscsdk_vectors_find(const char* id);

Source-derived role: SDK qualification platform helper for vectors find. Validate status, pointers and output length at each call.

27.23 QSCSDKTest/qscsdk_web_assets.h (3 declarations)

qscsdk_web_assets_index

bool qscsdk_web_assets_index(char** output, size_t* length);

Purpose: Embedded browser-shell and integrated Help assets for the SDK qualification workbench

qscsdk_web_assets_help

bool qscsdk_web_assets_help(char** output, size_t* length);

Source-derived role: SDK qualification platform helper for web assets help. Validate status, pointers and output length at each call.

qscsdk_web_assets_free

void qscsdk_web_assets_free(char* output);

Source-derived role: SDK qualification platform helper for web assets free. Validate status, pointers and output length at each call.

27.24 QSCSDKTest/qscsdk_web_icon.h (1 declarations)

qscsdk_web_icon_get

bool qscsdk_web_icon_get(const uint8_t** data, size_t* length);

Purpose: Returns the immutable embedded QSC SDK Test application icon

Parameters: data: Receives a pointer to the PNG icon bytes.; length: Receives the exact PNG byte length.

Return: Returns true when the output pointers are valid and the icon is available

27.25 QSCSDKTest/qscsdk_web_json.h (11 declarations)

qscsdk_web_json_catalog

bool qscsdk_web_json_catalog(char** output, size_t* output_len);

Purpose: In-memory JSON responses used by the localhost workbench API

qscsdk_web_json_summary

bool qscsdk_web_json_summary(char** output, size_t* output_len);

Source-derived role: Serialize a defined SDK workbench data object as an allocated JSON response; the caller must free the returned buffer via its matching API.

qscsdk_web_json_run_status

bool qscsdk_web_json_run_status(qscsdk_run_manager* manager, const char* run_id, char** output, size_t* output_len);

Source-derived role: Serialize a defined SDK workbench data object as an allocated JSON response; the caller must free the returned buffer via its matching API.

qscsdk_web_json_run_history

bool qscsdk_web_json_run_history(qscsdk_run_manager* manager, char** output, size_t* output_len);

Source-derived role: Serialize a defined SDK workbench data object as an allocated JSON response; the caller must free the returned buffer via its matching API.

qscsdk_web_json_run_result

bool qscsdk_web_json_run_result(qscsdk_run_manager* manager, const char* run_id, const char* test_id, char** output, size_t* output_len);

Source-derived role: Serialize a defined SDK workbench data object as an allocated JSON response; the caller must free the returned buffer via its matching API.

qscsdk_web_json_benchmarks

bool qscsdk_web_json_benchmarks(qscsdk_benchmark_manager* manager, char** output, size_t* output_len);

Source-derived role: Serialize a defined SDK workbench data object as an allocated JSON response; the caller must free the returned buffer via its matching API.

qscsdk_web_json_fuzz

bool qscsdk_web_json_fuzz(qscsdk_fuzz_manager* manager, char** output, size_t* output_len);

Source-derived role: Serialize a defined SDK workbench data object as an allocated JSON response; the caller must free the returned buffer via its matching API.

qscsdk_web_json_coverage

bool qscsdk_web_json_coverage(char** output, size_t* output_len);

Source-derived role: Serialize a defined SDK workbench data object as an allocated JSON response; the caller must free the returned buffer via its matching API.

qscsdk_web_json_reports

bool qscsdk_web_json_reports(const char* root, char** output, size_t* output_len);

Source-derived role: Serialize a defined SDK workbench data object as an allocated JSON response; the caller must free the returned buffer via its matching API.

qscsdk_web_json_report_bundle

bool qscsdk_web_json_report_bundle(const qscsdk_evidence_bundle* bundle, char** output, size_t* output_len);

Source-derived role: Serialize a defined SDK workbench data object as an allocated JSON response; the caller must free the returned buffer via its matching API.

qscsdk_web_json_free

void qscsdk_web_json_free(char* output);

Source-derived role: Serialize a defined SDK workbench data object as an allocated JSON response; the caller must free the returned buffer via its matching API.

27.26 QSCSDKTest/qscsdk_web_platform.h (9 declarations)

qscsdk_web_platform_initialize

bool qscsdk_web_platform_initialize(void);

Source-derived role: Perform initialize on the associated SDK state, preserving the caller’s ownership and native lifetime contract.

qscsdk_web_platform_shutdown

void qscsdk_web_platform_shutdown(void);

Source-derived role: SDK qualification platform helper for web platform shutdown. Validate status, pointers and output length at each call.

qscsdk_web_platform_listen

qscsdk_web_socket qscsdk_web_platform_listen(uint16_t requested_port, uint16_t* bound_port);

Source-derived role: SDK qualification platform helper for web platform listen. Validate status, pointers and output length at each call.

qscsdk_web_platform_accept

qscsdk_web_socket qscsdk_web_platform_accept(qscsdk_web_socket listener);

Source-derived role: SDK qualification platform helper for web platform accept. Validate status, pointers and output length at each call.

qscsdk_web_platform_receive

int32_t qscsdk_web_platform_receive(qscsdk_web_socket socket_handle, uint8_t* output, size_t output_len);

Source-derived role: SDK qualification platform helper for web platform receive. Validate status, pointers and output length at each call.

qscsdk_web_platform_send

bool qscsdk_web_platform_send(qscsdk_web_socket socket_handle, const uint8_t* input, size_t input_len);

Source-derived role: SDK qualification platform helper for web platform send. Validate status, pointers and output length at each call.

qscsdk_web_platform_close

void qscsdk_web_platform_close(qscsdk_web_socket socket_handle);

Source-derived role: Perform close on the associated SDK state, preserving the caller's ownership and native lifetime contract.

qscsdk_web_platform_random

bool qscsdk_web_platform_random(uint8_t* output, size_t output_len);

Source-derived role: SDK qualification platform helper for web platform random. Validate status, pointers and output length at each call.

qscsdk_web_platform_open_browser

bool qscsdk_web_platform_open_browser(const char* url);

Source-derived role: SDK qualification platform helper for web platform open browser. Validate status, pointers and output length at each call.

27.27 QSCSDKTest/qscsdk_web_server.h (1 declarations)

qscsdk_web_server_run

int32_t qscsdk_web_server_run(const qscsdk_web_server_options* options);

Source-derived role: SDK qualification platform helper for web server run. Validate status, pointers and output length at each call.

27.28 QSCSDKTest/qscsdk_web_session.h (4 declarations)

qscsdk_web_session_initialize

bool qscsdk_web_session_initialize(qscsdk_web_session* session);

Source-derived role: Perform initialize on the associated SDK state, preserving the caller's ownership and native lifetime contract.

qscsdk_web_session_exchange

bool qscsdk_web_session_exchange(qscsdk_web_session* session, const char* bootstrap_token);

Source-derived role: SDK qualification platform helper for web session exchange. Validate status, pointers and output length at each call.

qscsdk_web_session_validate

bool qscsdk_web_session_validate(const qscsdk_web_session* session, const char* session_token);

Source-derived role: SDK qualification platform helper for web session validate. Validate status, pointers and output length at each call.

qscsdk_web_session_dispose

void qscsdk_web_session_dispose(qscsdk_web_session* session);

Source-derived role: Perform dispose on the associated SDK state, preserving the caller's ownership and native lifetime contract.

27.29 QSCSDKTest/qscsdk_x509_tests.h (5 declarations)

qscsdk_x509_test_dns_match

qscsdk_test_status qscsdk_x509_test_dns_match(const qscsdk_test_context* context, char* detail, size_t detail_len);

Purpose: Qualify exact and wildcard DNS-name matching semantics

qscsdk_x509_test_time

qscsdk_test_status qscsdk_x509_test_time(const qscsdk_test_context* context, char* detail, size_t detail_len);

Purpose: Qualify RFC 5280 UTCTime and GeneralizedTime parsing and comparison

qscsdk_x509_test_pem_roundtrip

qscsdk_test_status qscsdk_x509_test_pem_roundtrip(const qscsdk_test_context* context, char* detail, size_t detail_len);

Purpose: Qualify certificate PEM wrapping and binary round-trip behavior

qscsdk_x509_test_spki_ec

qscsdk_test_status qscsdk_x509_test_spki_ec(const qscsdk_test_context* context, char* detail, size_t detail_len);

Purpose: Qualify EC SubjectPublicKeyInfo initialization and coordinate recovery

qscsdk_x509_test_spki_pqc

qscsdk_test_status qscsdk_x509_test_spki_pqc(const qscsdk_test_context* context, char* detail, size_t detail_len);

Purpose: Qualify active ML-DSA and ML-KEM SubjectPublicKeyInfo metadata and validation

27.30 QSCSDKTest/qscsdk_zip.h (1 declarations)

qscsdk_zip_create

bool qscsdk_zip_create(const char* output_path, const qscsdk_zip_entry* entries, size_t entry_count, char* detail, size_t detail_len);

Source-derived role: SDK qualification platform helper for zip create. Validate status, pointers and output length at each call.

Annex A: Complete Interactive Command Catalog

The following source-derived table enumerates all 393 recognized exact QSCSDKC console command strings. Commands are passed to the SDK example selector, not directly to the native cryptographic library. The parser does not define a stable interoperable network service API. The table retains literal spelling, order and whitespace; exact commands should be checked against sdkc.c if edited in a later revision.

Command family	Exact console command
-help	-help
-list	-list
-ecdh	-ecdh
-ecdh	-ecdh keypair
-ecdh	-ecdh shared
-ecdh	-ecdh derive
-ecdh	-ecdh roles
-ecdh	-ecdh publickey
-ecdh	-ecdh file
-ecdh	-ecdh verify
-ecdh	-ecdh negative
-eddh	-eddh
-eddh	-eddh keypair
-eddh	-eddh shared
-eddh	-eddh derive
-eddh	-eddh roles
-eddh	-eddh publickey
-eddh	-eddh file
-eddh	-eddh verify
-eddh	-eddh negative
-hqc	-hqc
-hqc	-hqc keypair
-hqc	-hqc encapsulate
-hqc	-hqc decapsulate
-hqc	-hqc derive
-hqc	-hqc roles
-hqc	-hqc publickey
-hqc	-hqc file
-hqc	-hqc verify
-hqc	-hqc negative
-mlkem	-mlkem
-mlkem	-mlkem keypair
-mlkem	-mlkem encapsulate
-mlkem	-mlkem decapsulate
-mlkem	-mlkem derive

Command family	Exact console command
-mlkem	-mlkem roles
-mlkem	-mlkem publickey
-mlkem	-mlkem file
-mlkem	-mlkem verify
-mlkem	-mlkem negative
-mceliece	-mceliece
-mceliece	-mceliece keypair
-mceliece	-mceliece encapsulate
-mceliece	-mceliece decapsulate
-mceliece	-mceliece derive
-mceliece	-mceliece roles
-mceliece	-mceliece publickey
-mceliece	-mceliece file
-mceliece	-mceliece verify
-mceliece	-mceliece negative
-mldsa	-mldsa
-mldsa	-mldsa keypair
-mldsa	-mldsa sign
-mldsa	-mldsa verify
-mldsa	-mldsa detached
-mldsa	-mldsa file
-mldsa	-mldsa publickey
-mldsa	-mldsa domain
-mldsa	-mldsa negative
-ecdsa	-ecdsa
-ecdsa	-ecdsa keypair
-ecdsa	-ecdsa sign
-ecdsa	-ecdsa verify
-ecdsa	-ecdsa detached
-ecdsa	-ecdsa file
-ecdsa	-ecdsa publickey
-ecdsa	-ecdsa domain
-ecdsa	-ecdsa negative
-eddsa	-eddsa
-eddsa	-eddsa keypair
-eddsa	-eddsa sign
-eddsa	-eddsa verify
-eddsa	-eddsa detached
-eddsa	-eddsa file
-eddsa	-eddsa publickey
-eddsa	-eddsa domain
-eddsa	-eddsa negative
-falcon	-falcon
-falcon	-falcon keypair

Command family	Exact console command
-falcon	-falcon sign
-falcon	-falcon verify
-falcon	-falcon detached
-falcon	-falcon file
-falcon	-falcon publickey
-falcon	-falcon domain
-falcon	-falcon negative
-sphincsplus	-sphincsplus
-sphincsplus	-sphincsplus keypair
-sphincsplus	-sphincsplus sign
-sphincsplus	-sphincsplus verify
-sphincsplus	-sphincsplus detached
-sphincsplus	-sphincsplus file
-sphincsplus	-sphincsplus publickey
-sphincsplus	-sphincsplus domain
-sphincsplus	-sphincsplus negative
-aes	-aes
-aes	-aes ecb
-aes	-aes cbc
-aes	-aes ctrbe
-aes	-aes ctrle
-aes	-aes gcm
-aes	-aes aad
-aes	-aes authfail
-aes	-aes file
-chacha	-chacha
-chacha	-chacha buffer
-chacha	-chacha stream
-chacha	-chacha poly1305
-chacha	-chacha aad
-chacha	-chacha authfail
-chacha	-chacha file
-csx	-csx
-csx	-csx buffer
-csx	-csx aad
-csx	-csx authfail
-csx	-csx file
-csx	-csx stream
-rcs	-rcs
-rcs	-rcs buffer
-rcs	-rcs aad
-rcs	-rcs authfail
-rcs	-rcs file
-rcs	-rcs stream

Command family	Exact console command
-sha2	-sha2
-sha2	-sha2 256
-sha2	-sha2 384
-sha2	-sha2 512
-sha2	-sha2 short
-sha2	-sha2 long
-sha2	-sha2 stream
-sha2	-sha2 file
-sha3	-sha3
-sha3	-sha3 128
-sha3	-sha3 256
-sha3	-sha3 512
-sha3	-sha3 short
-sha3	-sha3 long
-sha3	-sha3 stream
-sha3	-sha3 file
-csg	-csg
-csg	-csg 256
-csg	-csg 512
-csg	-csg generate
-csg	-csg info
-csg	-csg update
-csg	-csg stream
-csg	-csg file
-csg	-csg verify
-csg	-csg lifecycle
-hcg	-hcg
-hcg	-hcg 512
-hcg	-hcg generate
-hcg	-hcg info
-hcg	-hcg update
-hcg	-hcg stream
-hcg	-hcg file
-hcg	-hcg verify
-hcg	-hcg lifecycle
-scb	-scb
-scb	-scb 256
-scb	-scb 512
-scb	-scb generate
-scb	-scb info
-scb	-scb update
-scb	-scb stream
-scb	-scb file
-scb	-scb verify

Command family	Exact console command
-hkdf	-hkdf
-hkdf	-hkdf 256
-hkdf	-hkdf 512
-hkdf	-hkdf derive
-hkdf	-hkdf extract
-hkdf	-hkdf keys
-hkdf	-hkdf info
-hkdf	-hkdf verify
-hkdf	-hkdf file
-shake	-shake
-shake	-shake 128
-shake	-shake 256
-shake	-shake 512
-shake	-shake generate
-shake	-shake stream
-shake	-shake keys
-shake	-shake file
-shake	-shake verify
-cshake	-cshake
-cshake	-cshake 128
-cshake	-cshake 256
-cshake	-cshake 512
-cshake	-cshake domain
-cshake	-cshake keys
-cshake	-cshake file
-cshake	-cshake verify
-hmac	-hmac
-hmac	-hmac 256
-hmac	-hmac 512
-hmac	-hmac stream
-hmac	-hmac file
-hmac	-hmac verify
-kmac	-kmac
-kmac	-kmac 256
-kmac	-kmac 512
-kmac	-kmac stream
-kmac	-kmac file
-kmac	-kmac verify
-pbmac1	-pbmac1
-pbmac1	-pbmac1 256
-pbmac1	-pbmac1 512
-pbmac1	-pbmac1 stream
-pbmac1	-pbmac1 file
-pbmac1	-pbmac1 verify

Command family	Exact console command
-qmac	-qmac
-qmac	-qmac compute
-qmac	-qmac stream
-qmac	-qmac file
-qmac	-qmac verify
-secrand	-secrand
-secrand	-secrand 256
-secrand	-secrand generate
-secrand	-secrand keys
-secrand	-secrand values
-secrand	-secrand bounded
-secrand	-secrand info
-secrand	-secrand stream
-secrand	-secrand file
-secrand	-secrand verify
-secrand	-secrand lifecycle
-acp	-acp
-acp	-acp generate
-acp	-acp keys
-acp	-acp values
-acp	-acp file
-acp	-acp limit
-acp	-acp verify
-cjp	-cjp
-cjp	-cjp available
-cjp	-cjp generate
-cjp	-cjp keys
-cjp	-cjp values
-cjp	-cjp limit
-cjp	-cjp verify
-csp	-csp
-csp	-csp generate
-csp	-csp keys
-csp	-csp values
-csp	-csp limit
-csp	-csp verify
-rdp	-rdp
-rdp	-rdp available
-rdp	-rdp generate
-rdp	-rdp keys
-rdp	-rdp values
-rdp	-rdp limit
-rdp	-rdp verify
-collection	-collection

Command family	Exact console command
-collection	-collection keyed
-collection	-collection remove
-collection	-collection serialize
-collection	-collection list
-collection	-collection listops
-collection	-collection queue
-collection	-collection array
-collection	-collection negative
-cpuid	-cpuid
-cpuid	-cpuid features
-cpuid	-cpuid vendor
-cpuid	-cpuid crypto
-cpuid	-cpuid simd
-cpuid	-cpuid cache
-cpuid	-cpuid topology
-cpuid	-cpuid rdrand
-cpuid	-cpuid platform
-cpuid	-cpuid print
-encoding	-encoding
-encoding	-encoding hex
-encoding	-encoding base64
-encoding	-encoding pem
-encoding	-encoding ber
-encoding	-encoding roundtrip
-encoding	-encoding sizes
-encoding	-encoding file
-encoding	-encoding verify
-fileutils	-fileutils
-fileutils	-fileutils path
-fileutils	-fileutils write
-fileutils	-fileutils read
-fileutils	-fileutils append
-fileutils	-fileutils copy
-fileutils	-fileutils info
-fileutils	-fileutils folder
-fileutils	-fileutils enumerate
-fileutils	-fileutils remove
-intutils	-intutils
-intutils	-intutils be
-intutils	-intutils le
-intutils	-intutils increment
-intutils	-intutils rotate
-intutils	-intutils bit
-intutils	-intutils verify

Command family	Exact console command
-intutils	-intutils mask
-intutils	-intutils hex
-intutils	-intutils clear
-intutils	-intutils math
-memutils	-memutils
-memutils	-memutils copy
-memutils	-memutils move
-memutils	-memutils clear
-memutils	-memutils secureerase
-memutils	-memutils compare
-memutils	-memutils uniform
-memutils	-memutils xor
-memutils	-memutils alloc
-memutils	-memutils aligned
-memutils	-memutils secure
-memutils	-memutils zeroed
-memutils	-memutils page
-stringutils	-stringutils
-stringutils	-stringutils length
-stringutils	-stringutils copy
-stringutils	-stringutils concat
-stringutils	-stringutils compare
-stringutils	-stringutils find
-stringutils	-stringutils parse
-stringutils	-stringutils format
-stringutils	-stringutils case
-stringutils	-stringutils numeric
-stringutils	-stringutils hex
-socketutils	-socketutils
-socketutils	-socketutils start
-socketutils	-socketutils address
-socketutils	-socketutils create
-socketutils	-socketutils options
-socketutils	-socketutils blocking
-socketutils	-socketutils tcp
-socketutils	-socketutils udp
-socketutils	-socketutils ready
-socketutils	-socketutils negative
-timestamp	-timestamp
-timestamp	-timestamp current
-timestamp	-timestamp epoch
-timestamp	-timestamp convert
-timestamp	-timestamp compare
-timestamp	-timestamp utc

Command family	Exact console command
-timestamp	-timestamp stopwatch
-timestamp	-timestamp timerex
-timestamp	-timestamp format
-tls	-tls
-tls	-tls certificate
-tls	-tls client
-tls	-tls server
-tls	-tls loopback
-tls	-tls echo
-tls	-tls hostname
-tls	-tls mtlscert
-tls	-tls alpn
-tls	-tls sni
-tls	-tls groups
-tls	-tls fragments
-tls	-tls timeout
-tls	-tls shutdown
-tls	-tls negative
-tls	-tls live
-tls	-tls live loopback
-tls	-tls live echo
-tls	-tls live alpn
-tls	-tls live sni
-tls	-tls live mtls
-tls	-tls live shutdown
-tls	-tls live negative
-x509	-x509
-x509	-x509 parse
-x509	-x509 inspect
-x509	-x509 root
-x509	-x509 chain
-x509	-x509 hostname
-x509	-x509 validity
-x509	-x509 usage
-x509	-x509 fingerprint
-x509	-x509 publickey
-x509	-x509 sign
-x509	-x509 verify
-x509	-x509 negative
-x509	-x509 author
-x509	-x509 author builder
-x509	-x509 author root
-x509	-x509 author csr
-x509	-x509 author issue

Command family	Exact console command
-x509	-x509 author pem
-x509	-x509 author key
-x509	-x509 author profile
-x509	-x509 author negative
-run	-run all
-quit	-quit

Table A.1 Full interactive command-string inventory

Annex B: Registered Qualification Tests

The 26 immutable registry entries below are described by source-derived IDs, functional category and display title. A test ID is a persisted evidence key; it must remain stable or be explicitly versioned when changes break semantics. Where vector execution selects a single native compile-time parameter set, a registered case does not imply coverage of every parameter-set file in the archive.

Test ID	Class	Source title
QSC.SELFTEST.AES	selftest	AES internal self-test
QSC.SELFTEST.CHACHA	selftest	ChaCha internal self-test
QSC.SELFTEST.CSX	selftest	CSX internal self-test
QSC.SELFTEST.RCS	selftest	RCS internal self-test
QSC.SELFTEST.SHA2	selftest	SHA-2 internal self-test
QSC.SELFTEST.SHA3	selftest	SHA-3 internal self-test
CRYPTO.MLKEM.OPERATIONS	crypto	ML-KEM operations and implicit rejection
CRYPTO.MLDSA.SIGNATURE	crypto	ML-DSA signing and tamper rejection
CRYPTO.SLHDSA.SIGNATURE	crypto	SLH-DSA signing and tamper rejection
CRYPTO.ECDH.EXCHANGE	crypto	ECDH reciprocal shared-secret exchange
CRYPTO.ECDSA.SIGNATURE	crypto	ECDSA signing and tamper rejection
CRYPTO.EDDSA.SIGNATURE	crypto	EdDSA signing and tamper rejection
TLS13.CODEC.PRIMITIVES	tls	TLS 1.3 codec primitives and bounds
TLS13.EXTENSIONS.CORE	tls	TLS 1.3 supported_versions and SNI extensions
TLS13.TRANSCRIPT.SHA256	tls	TLS 1.3 SHA-256 transcript
TLS13.RECORD.PLAINTEXT	tls	TLS 1.3 plaintext record framing
TLS13.KEYSCHEDULE.FINISHED	tls	TLS 1.3 Finished verification
TLS13.PQ.CAPABILITIES	tls	TLS post-quantum capability metadata
ASSURANCE.VECTORS.INTEGRITY	assurance	Packaged NIST vector corpus integrity
CAVP.AES.CBC.KAT	vectors	AES CBC NIST known-answer vectors
CAVP.SHA2.SHA256.SHORT	vectors	SHA-256 NIST short-message vectors
CAVP.SHA3.SHA3_256.SHORT	vectors	SHA3-256 NIST short-message vectors
ACVP.MLKEM.KEYGEN	vectors	ML-KEM NIST key-generation vectors
ACVP.MLKEM.ENCAP	vectors	ML-KEM NIST encapsulation vectors
ACVP.MLDSA.KEYGEN	vectors	ML-DSA NIST key-generation vectors
ACVP.SLHDSA.KEYGEN	vectors	SLH-DSA NIST key-generation vectors

Table B.1 Test catalog

Annex C: Packaged Vector Corpus

Each row below represents a manifest-declared response-vector file. Count is the manifest case count, not an executed case count. SHA-256 and file size of every local fixture matched the manifest during this documentation task. The complete per-file digest remains in Vectors/manifest.json and should be rechecked whenever the package changes.

Algorithm family	Local .rsp file	Cases	Bytes
AES	CBCGFSbox128.rsp	14	2787
AES	CBCGFSbox256.rsp	10	2367
AES	CBCKeySbox128.rsp	42	7990
AES	CBCKeySbox256.rsp	32	7154
AES	CBCMCT128.rsp	200	37376
AES	CBCMCT256.rsp	200	43776
AES	CBCMMT128.rsp	20	9654
AES	CBCMMT256.rsp	20	10294
AES	CBCVarKey128.rsp	256	47849
AES	CBCVarKey256.rsp	512	112105
AES	CBCVarTxt128.rsp	256	47849
AES	CBCVarTxt256.rsp	256	56041
AES	CTRVarKey128.rsp	4	980
AES	CTRVarKey256.rsp	4	1101
AES	ECBGFSbox128.rsp	14	2241
AES	ECBGFSbox256.rsp	10	1977
AES	ECBKeySbox128.rsp	42	6352
AES	ECBKeySbox256.rsp	32	5906
AES	ECBMCT128.rsp	200	29576
AES	ECBMCT256.rsp	200	35976
AES	ECBMMT128.rsp	20	8874
AES	ECBMMT256.rsp	20	9514
AES	ECBVarKey128.rsp	256	37865
AES	ECBVarKey256.rsp	512	92137
AES	ECBVarTxt128.rsp	256	37865
AES	ECBVarTxt256.rsp	256	46057
AES	GCMVarKey128.rsp	15	3963
AES	GCMVarKey256.rsp	15	4441
MLDSA	MLDSA44KeyGen.rsp	25	196331
MLDSA	MLDSA44SigGen.rsp	15	305148
MLDSA	MLDSA65KeyGen.rsp	25	301940
MLDSA	MLDSA65SigGen.rsp	15	412151
MLDSA	MLDSA87KeyGen.rsp	25	377140
MLDSA	MLDSA87SigGen.rsp	15	516545
MLKEM	MLKEMEncap1024.rsp	25	320174

Algorithm family	Local .rsp file	Cases	Bytes
MLKEM	MLKEMEncap512.rsp	25	164974
MLKEM	MLKEMEncap768.rsp	25	238574
MLKEM	MLKEMKeyGen1024.rsp	25	241395
MLKEM	MLKEMKeyGen512.rsp	24	124531
MLKEM	MLKEMKeyGen768.rsp	24	183795
SHA2	HMAC.rsp	1575	849996
SHA2	HMACSHA256.rsp	225	108489
SHA2	HMACSHA512.rsp	375	250344
SHA2	SHA256LongMsg.rsp	64	426209
SHA2	SHA256Monte.rsp	100	8751
SHA2	SHA256ShortMsg.rsp	65	10299
SHA2	SHA384LongMsg.rsp	128	1683749
SHA2	SHA384Monte.rsp	100	11983
SHA2	SHA384ShortMsg.rsp	129	32672
SHA2	SHA512LongMsg.rsp	128	1687845
SHA2	SHA512Monte.rsp	100	15215
SHA2	SHA512ShortMsg.rsp	129	36800
SHA3	SHA3_256LongMsg.rsp	100	1420540
SHA3	SHA3_256Monte.rsp	100	8801
SHA3	SHA3_256ShortMsg.rsp	137	31406
SHA3	SHA3_512LongMsg.rsp	100	767722
SHA3	SHA3_512Monte.rsp	100	15265
SHA3	SHA3_512ShortMsg.rsp	73	16802
SHA3	SHAKE128LongMsg.rsp	100	1747365
SHA3	SHAKE128Monte.rsp	100	20135
SHA3	SHAKE128ShortMsg.rsp	337	135243
SHA3	SHAKE128VariableOut.rsp	1126	269703
SHA3	SHAKE256LongMsg.rsp	100	1420945
SHA3	SHAKE256Monte.rsp	100	31170
SHA3	SHAKE256ShortMsg.rsp	273	100843
SHA3	SHAKE256VariableOut.rsp	1246	458545
SLHDSA	SLHDSASHAKE128KeyGen.rsp	10	3435
SLHDSA	SLHDSASHAKE128SigGen.rsp	7	183342
SLHDSA	SLHDSASHAKE192KeyGen.rsp	10	4875
SLHDSA	SLHDSASHAKE192SigGen.rsp	7	303684
SLHDSA	SLHDSASHAKE256KeyGen.rsp	10	6317
SLHDSA	SLHDSASHAKE256SigGen.rsp	7	487394

Table C.1 Verified vector fixture index

Annex D: Public Types, Enums and Constants

These types define memory ownership and ABI layouts within the standalone SDK executables. Their descriptions are relevant to contributors, not a guarantee of binary-stable application integration. The SDK source may change these layouts independently of QSC or QSCABI.

Source typedef	Kind	Header
qscsdk_benchmark_state	enum	QSCSDKTest/qscsdk_benchmark.h
qscsdk_benchmark_definition	struct	QSCSDKTest/qscsdk_benchmark.h
qscsdk_benchmark_result	struct	QSCSDKTest/qscsdk_benchmark.h
qscsdk_benchmark_capabilities	struct	QSCSDKTest/qscsdk_benchmark.h
qscsdk_benchmark_snapshot	struct	QSCSDKTest/qscsdk_benchmark.h
qscsdk_benchmark_manager	struct	QSCSDKTest/qscsdk_benchmark.h
qscsdk_cli_action	enum	QSCSDKTest/qscsdk_cli.h
qscsdk_cli_options	struct	QSCSDKTest/qscsdk_cli.h
qscsdk_test_status	enum	QSCSDKTest/qscsdk_common.h
qscsdk_test_type	enum	QSCSDKTest/qscsdk_common.h
qscsdk_assertion_status	enum	QSCSDKTest/qscsdk_common.h
qscsdk_evidence_flag	enum	QSCSDKTest/qscsdk_common.h
qscsdk_standard_reference	struct	QSCSDKTest/qscsdk_common.h
qscsdk_vector_source	struct	QSCSDKTest/qscsdk_common.h
qscsdk_assertion_record	struct	QSCSDKTest/qscsdk_common.h
qscsdk_artifact_record	struct	QSCSDKTest/qscsdk_common.h
qscsdk_run_environment	struct	QSCSDKTest/qscsdk_common.h
qscsdk_test_context	struct	QSCSDKTest/qscsdk_common.h
qscsdk_test_definition	struct	QSCSDKTest/qscsdk_common.h
qscsdk_run_summary	struct	QSCSDKTest/qscsdk_common.h
qscsdk_coverage_component	struct	QSCSDKTest/qscsdk_coverage.h
qscsdk_coverage_snapshot	struct	QSCSDKTest/qscsdk_coverage.h
qscsdk_evidence_bundle	struct	QSCSDKTest/qscsdk_evidence.h
qscsdk_fuzz_state	enum	QSCSDKTest/qscsdk_fuzz.h
qscsdk_fuzz_target	struct	QSCSDKTest/qscsdk_fuzz.h
qscsdk_fuzz_snapshot	struct	QSCSDKTest/qscsdk_fuzz.h
qscsdk_fuzz_manager	struct	QSCSDKTest/qscsdk_fuzz.h
qscsdk_run_state	enum	QSCSDKTest/qscsdk_run_manager.h
qscsdk_run_log_record	struct	QSCSDKTest/qscsdk_run_manager.h
qscsdk_run_status_snapshot	struct	QSCSDKTest/qscsdk_run_manager.h
qscsdk_run_result_brief	struct	QSCSDKTest/qscsdk_run_manager.h
qscsdk_run_history_record	struct	QSCSDKTest/qscsdk_run_manager.h
qscsdk_run_manager	struct	QSCSDKTest/qscsdk_run_manager.h
qscsdk_web_server_options	struct	QSCSDKTest/qscsdk_web_server.h
qscsdk_web_session	struct	QSCSDKTest/qscsdk_web_session.h
qscsdk_zip_entry	struct	QSCSDKTest/qscsdk_zip.h

Table D.1 Named source-defined types

D.1 Stable values and project limits

Name	Value	Role
QSCSDK_WEB_DEFAULT_PORT	59671	Default loopback HTTP port.

Name	Value	Role
QSCSDK_RUN_HISTORY_MAX	8	Bounded run-history record count.
QSCSDK_RUN_LOG_MAX	256	Maximum in-memory run log records.
QSCSDK_RUN_LOG_TEXT_MAX	192	Maximum bytes in one run log text field.
QSCSDK_SCHEMA_VERSION	qsc-sdk-test-report-v2	Structured report schema identifier.
QSCSDK_CATALOG_SCHEMA_VERSION	qsc-sdk-test-catalog-v1	Immutable catalog schema identifier.
QSCSDK_EVIDENCE_SCHEMA_VERSION	qsc-sdk-evidence-bundle-v3	Evidence export manifest schema.
QSCSDK_DEFAULT_SEED	0x51534353444B0001	Deterministic default run seed.

Table D.2 Key source-defined constants

All source/header files in the supplied ZIP remain unchanged. No builds, runtime cryptographic tests or live network checks were performed during authoring. This specification does not supersede QSC native API documentation or QRCS-CASL.